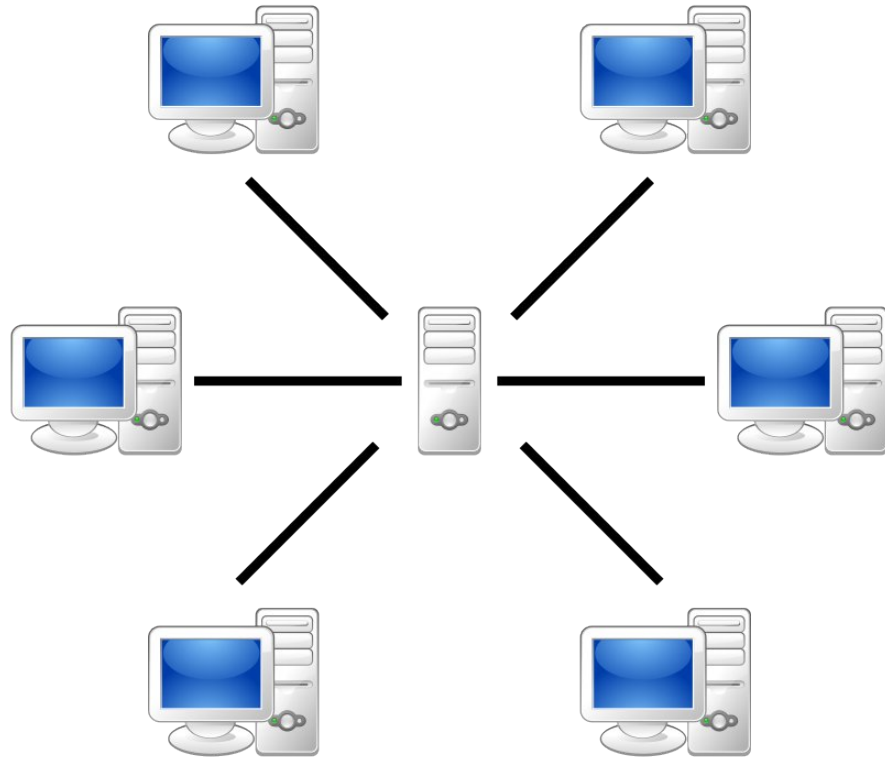




Informatik 2: Grundlagen Netzwerk & -programmierung



Client / Server



Server

Ein Server ist ein Gerät oder Programm (Prozess), das mit einem anderen Programm, dem Client, kommuniziert, um ihm Zugang zu einem Dienst zu verschaffen.

Client

Ein Client kann einen Dienst bei dem Server anfordern, der diesen Dienst bereitstellt.

Protokoll

Kommunikationsprotokoll, eine Vereinbarung wie die Datenübertragung zwischen den beiden Parteien Server und Client abläuft.

Request (deutsch: Anfrage)

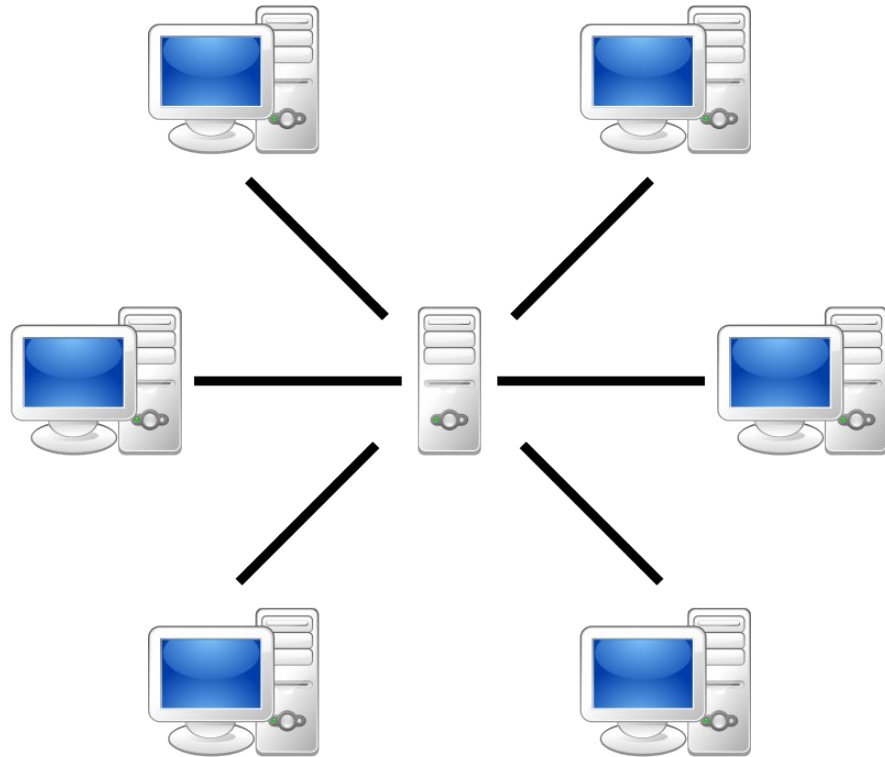
Anforderung eines Clients an den Server, dessen Dienst er benötigt.

Response (deutsch: Antwort)

Antwort eines Servers auf eine Anforderung eines Clients.

<https://de.wikipedia.org/wiki/Client-Server-Modell>

Client / Server



Server

Ein Server ist ein Gerät oder Programm (Prozess), das mit einem anderen Programm, dem Client, kommuniziert, um ihm Zugang zu einem Dienst zu verschaffen. „Server“ ist eine Rolle, nicht unbedingt ein Computer. **Ein Computer kann Server und Client zugleich sein (Peer-to-Peer).**

Client

Ein Client kann einen Dienst bei dem Server anfordern, der diesen Dienst bereitstellt.

Protokoll

Kommunikationsprotokoll, eine Vereinbarung wie die Datenübertragung zwischen den beiden Parteien Server und Client abläuft.

Request (deutsch: Anfrage)

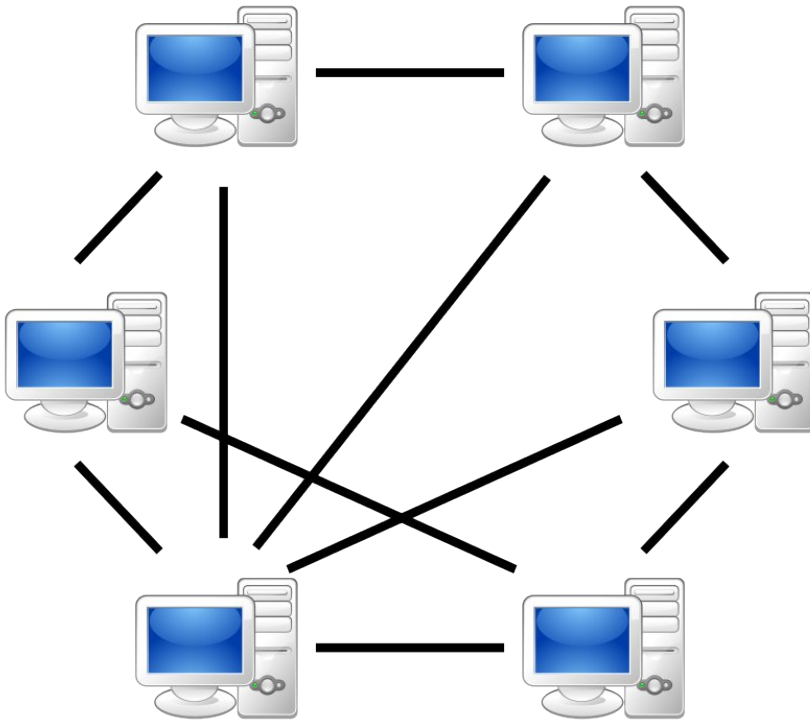
Anforderung eines Clients an den Server, dessen Dienst er benötigt.

Response (deutsch: Antwort)

Antwort eines Servers auf eine Anforderung eines Clients.

<https://de.wikipedia.org/wiki/Client-Server-Modell>

Peer-to-Peer (P2P)

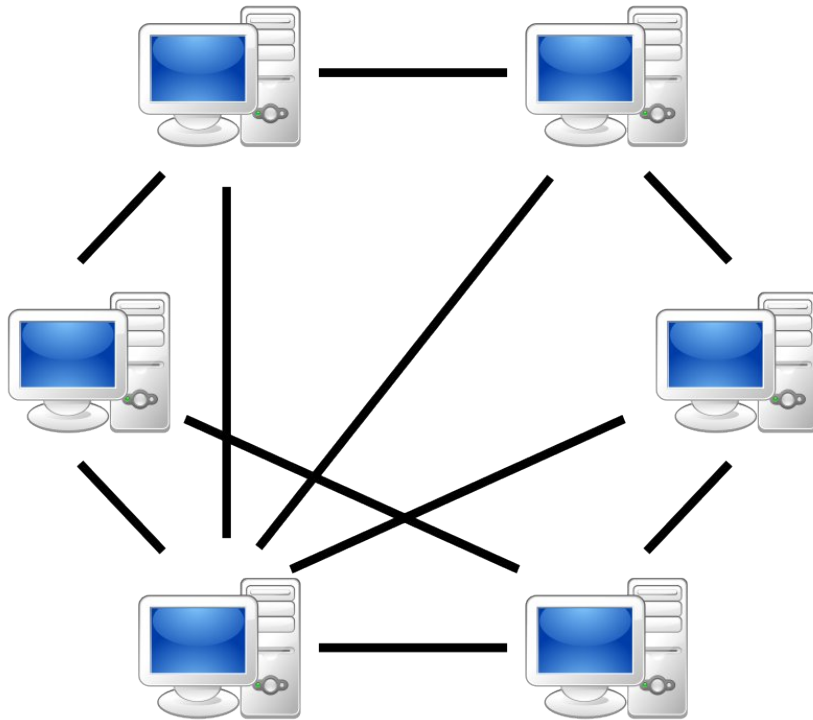


Ein Computer kann Server und Client zugleich sein (Peer-to-Peer):

- Peer-to-Peer (P2P) ist eine dezentrale Kommunikation (jeder P2P-Knoten hat dieselben Funktionen verfügt und kann eine Kommunikationssitzung starten).
- Jeder Knoten kann sowohl als Client als auch als Server arbeiten. (Im Gegensatz zum Client-Server-Modell)
- P2P-Systeme werden verwendet für
 - anonymisiertes Routing des Netzwerkverkehrs
 - massive parallele Computerumgebungen
 - verteilten Speicher (z.B. Blockchain)

<https://de.wikipedia.org/wiki/Peer-to-Peer>
<https://de.wikipedia.org/wiki/Napster>
https://de.wikipedia.org/wiki/Shawn_Fanning

Peer-to-Peer (P2P)

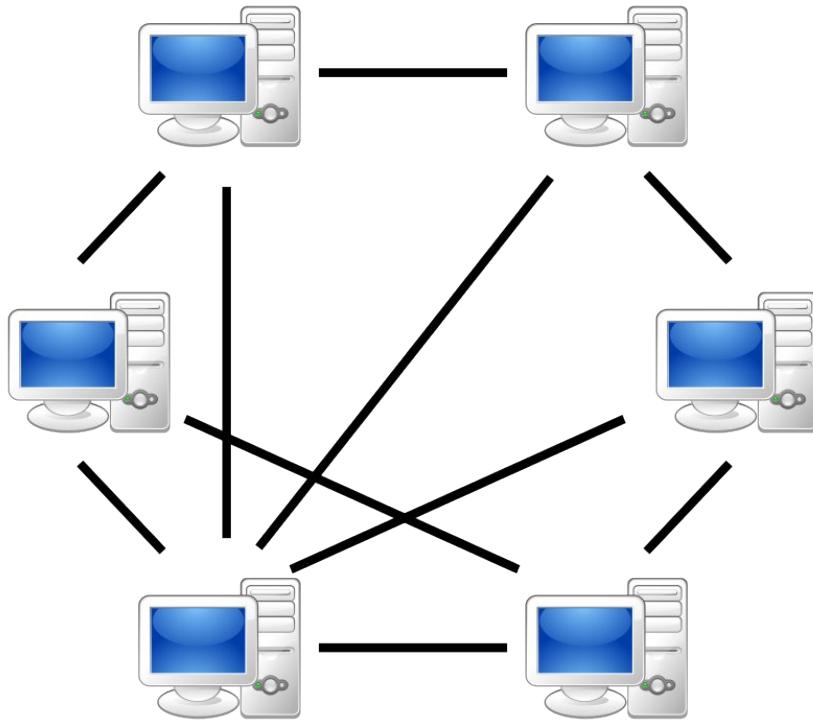


Ein Computer kann Server und Client zugleich sein (Peer-to-Peer) [1]:

- Peer-to-Peer (P2P) ist eine dezentrale Kommunikation (jeder P2P-Knoten hat dieselben Funktionen verfügt und kann eine Kommunikationssitzung starten).
- Jeder Knoten kann sowohl als Client als auch als Server arbeiten. (Im Gegensatz zum Client-Server-Modell)
- P2P-Systeme werden verwendet für
 - anonymisiertes Routing des Netzwerkverkehrs
 - massive parallele Computerumgebungen
 - verteilten Speicher (z.B. Blockchain)
- dezentrale Struktur von P2P-Systemen: widerstandsfähig gegen Cyber-Angriffe und zudem skalierbar. Je mehr Benutzer desto robuster und skalierbarer wird es [2].

[1] <https://www.heise.de/tipps-tricks/Peer-to-Peer-was-ist-das-4660681.html>
[2] <https://www.btc-echo.de/academy/bibliothek/peer-to-peer-p2p/>
[3] <https://jonathan.overholt.org/tutorial/winsock>
[4] Patricia Fusco, The Napster Nightmare (2000)
<https://web.archive.org/web/20111019152028/http://www.isp-planet.com/politics/napster.html>

Peer-to-Peer (P2P)

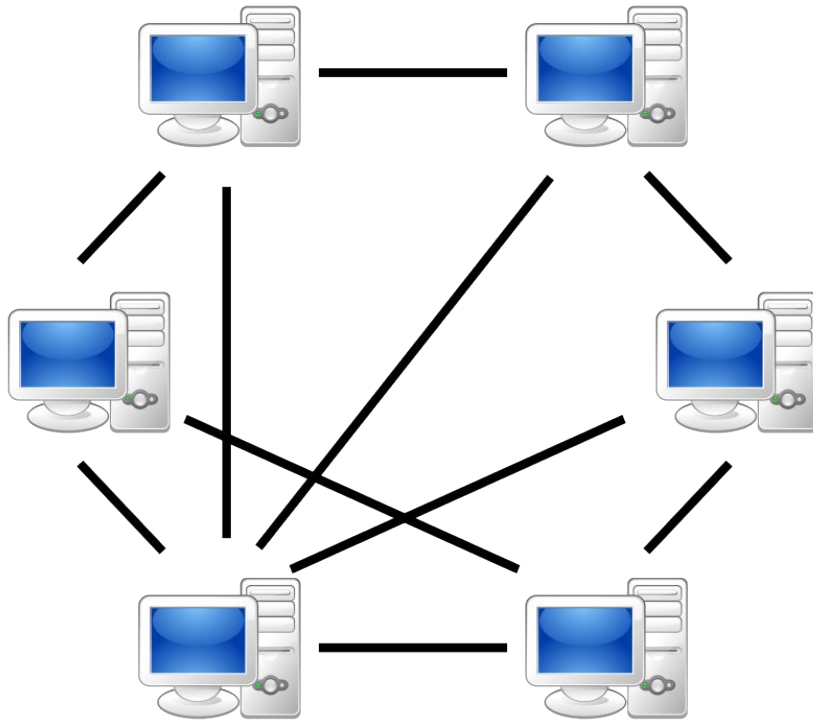


Ein Computer kann Server und Client zugleich sein (Peer-to-Peer) [1]:

- Peer-to-Peer (P2P) ist eine dezentrale Kommunikation (jeder P2P-Knoten hat dieselben Funktionen verfügt und kann eine Kommunikationssitzung starten).
- Jeder Knoten kann sowohl als Client als auch als Server arbeiten. (Im Gegensatz zum Client-Server-Modell)
- P2P-Systeme werden verwendet für
 - anonymisiertes Routing des Netzwerkverkehrs
 - massive parallele Computerumgebungen
 - verteilten Speicher (z.B. Blockchain)
- dezentrale Struktur von P2P-Systemen: widerstandsfähig gegen Cyber-Angriffe und zudem skalierbar. Je mehr Benutzer desto robuster und skalierbarer wird es [2].
- Einige komplexe Probleme: z.B. routing loops, Firewalls, load balancing und link utilization.
- P2P können die allgemeine Netzwerk Performance reduzieren (im Besonderen wenn sie fehlerhaft implementiert sind) [3]

[1] <https://www.heise.de/tipps-tricks/Peer-to-Peer-was-ist-das-4660681.html>
[2] <https://www.btc-echo.de/academy/bibliothek/peer-to-peer-p2p/>
[3] <https://jonathan.overholt.org/tutorial/winsock>
[4] Patricia Fusco, The Napster Nightmare (2000)
<https://web.archive.org/web/20111019152028/http://www.isp-planet.com/politics/napster.html>

Peer-to-Peer (P2P)



Ein Computer kann Server und Client zugleich sein (Peer-to-Peer) [1]:

- Peer-to-Peer (P2P) ist eine dezentrale Kommunikation (jeder P2P-Knoten hat dieselben Funktionen verfügt und kann eine Kommunikationssitzung starten).
- Jeder Knoten kann sowohl als Client als auch als Server arbeiten. (Im Gegensatz zum Client-Server-Modell)
- P2P-Systeme werden verwendet für
 - anonymisiertes Routing des Netzwerkverkehrs
 - massive parallele Computerumgebungen
 - verteilten Speicher (z.B. Blockchain)
- dezentrale Struktur von P2P-Systemen: widerstandsfähig gegen Cyber-Angriffe und zudem skalierbar. Je mehr Benutzer desto robuster und skalierbarer wird es [2].
- Einige komplexe Probleme: z.B. routing loops, Firewalls, load balancing und link utilization.
- P2P können die allgemeine Netzwerk Performance reduzieren (im Besonderen wenn sie fehlerhaft implementiert sind) [3]
- Die meisten P2P-Programme konzentrieren sich auf die Verteilung von Medien (z.B. Video- und Musik-Dateien). Daher wird P2P häufig mit Softwarepiraterie und Urheberrechtsverletzungen in Verbindung gebracht.*



*zeitweise waren 61% des externen Internetverkehrs von US-Colleges MP3 Dateiübertragungen. [4]

[1] <https://www.heise.de/tipps-tricks/Peer-to-Peer-was-ist-das-4660681.html>

[2] <https://www.btc-echo.de/academy/bibliothek/peer-to-peer-p2p/>

[3] <https://jonathan.overholt.org/tutorial/winsock>

[4] Patricia Fusco, The Napster Nightmare (2000)

<https://web.archive.org/web/20111019152028/http://www.isp-planet.com/politics/napster.html>

Internetkommunikation: ISO/OSI-Referenzmodell



OSI: *Open Systems Interconnection model*

OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA TLS, SOCKS
Darstellung (6)		
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

Internetkommunikation: ISO/OSI-Referenzmodell



OSI: *Open Systems Interconnection model*

Vorlesung ↑	OSI-Schicht	TCP/IP-Schicht	Beispiel	
	Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA	(Application Layer)
	Darstellung (6)		TLS, SOCKS	(Presentation Layer)
	Sitzung (5)			(Session Layer)
	Transport (4)	Transport	TCP, UDP, SCTP	(Transport Layer)
	Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)	(Network Layer)
	Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI	(Data Link Layer)
	Bitübertragung (1)			(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>

<https://de.wikipedia.org/wiki/OSI-Modell>

Internetkommunikation: ISO/OSI-Referenzmodell

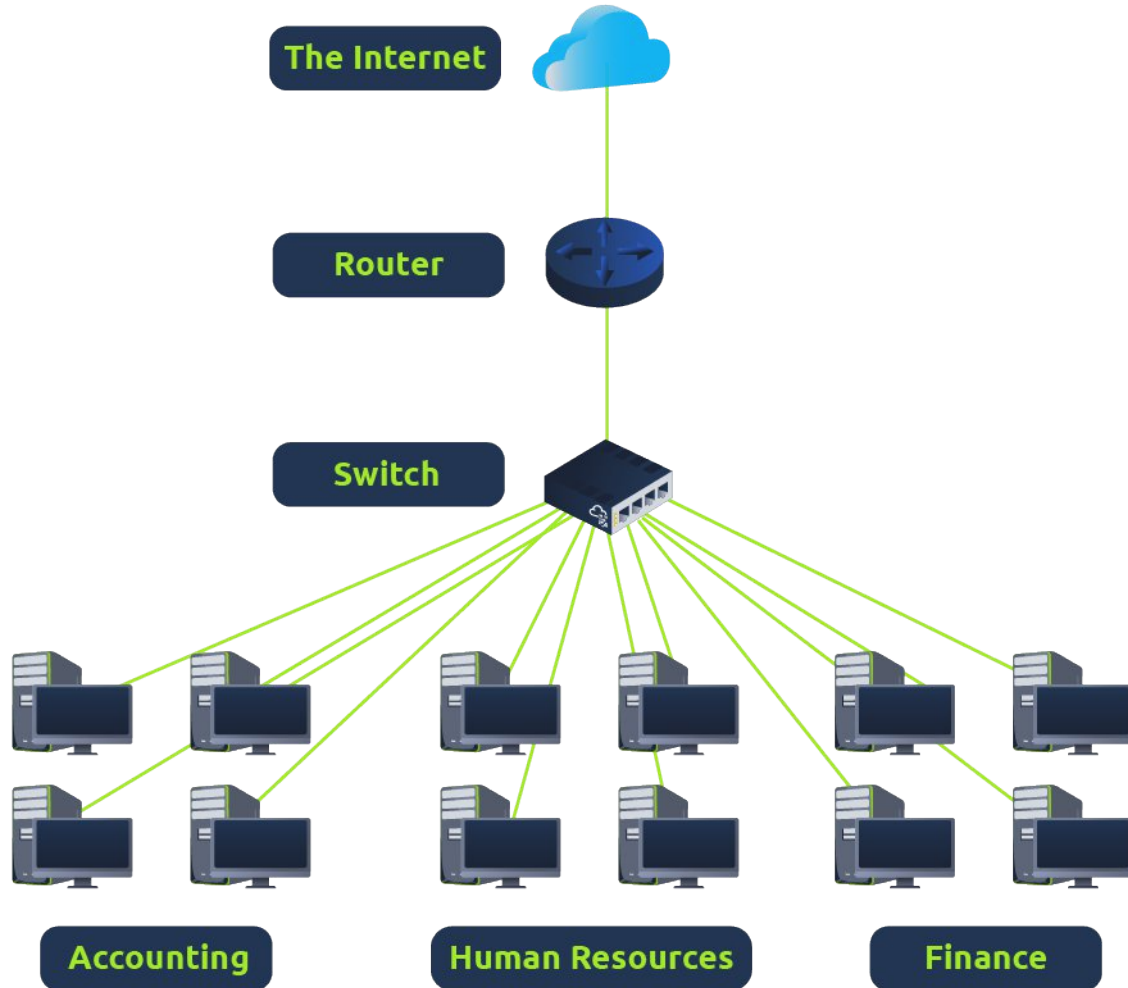


OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA TLS, SOCKS
Darstellung (6)		
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

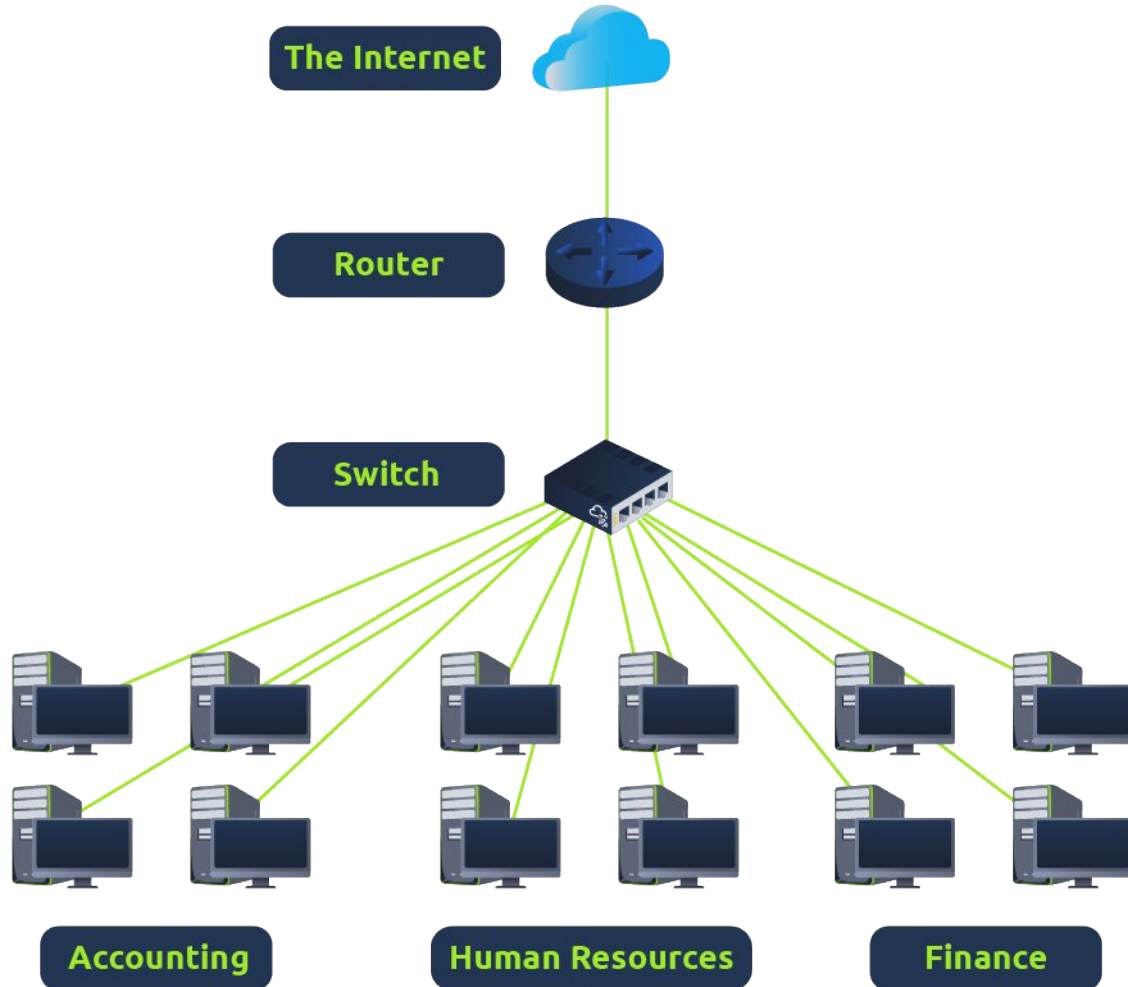
Switch und Router



Switch

- ermöglicht die gemeinsame Nutzung von Ressourcen, indem alle Geräte, z.B. Computer, Drucker und Server, in einem Netzwerk miteinander verbunden werden.
- Die verbundenen Geräte können über den Switch Informationen teilen und miteinander kommunizieren, unabhängig davon, wo in einem Gebäude oder wo auf einem Campus sie sich gerade befinden.

Switch und Router

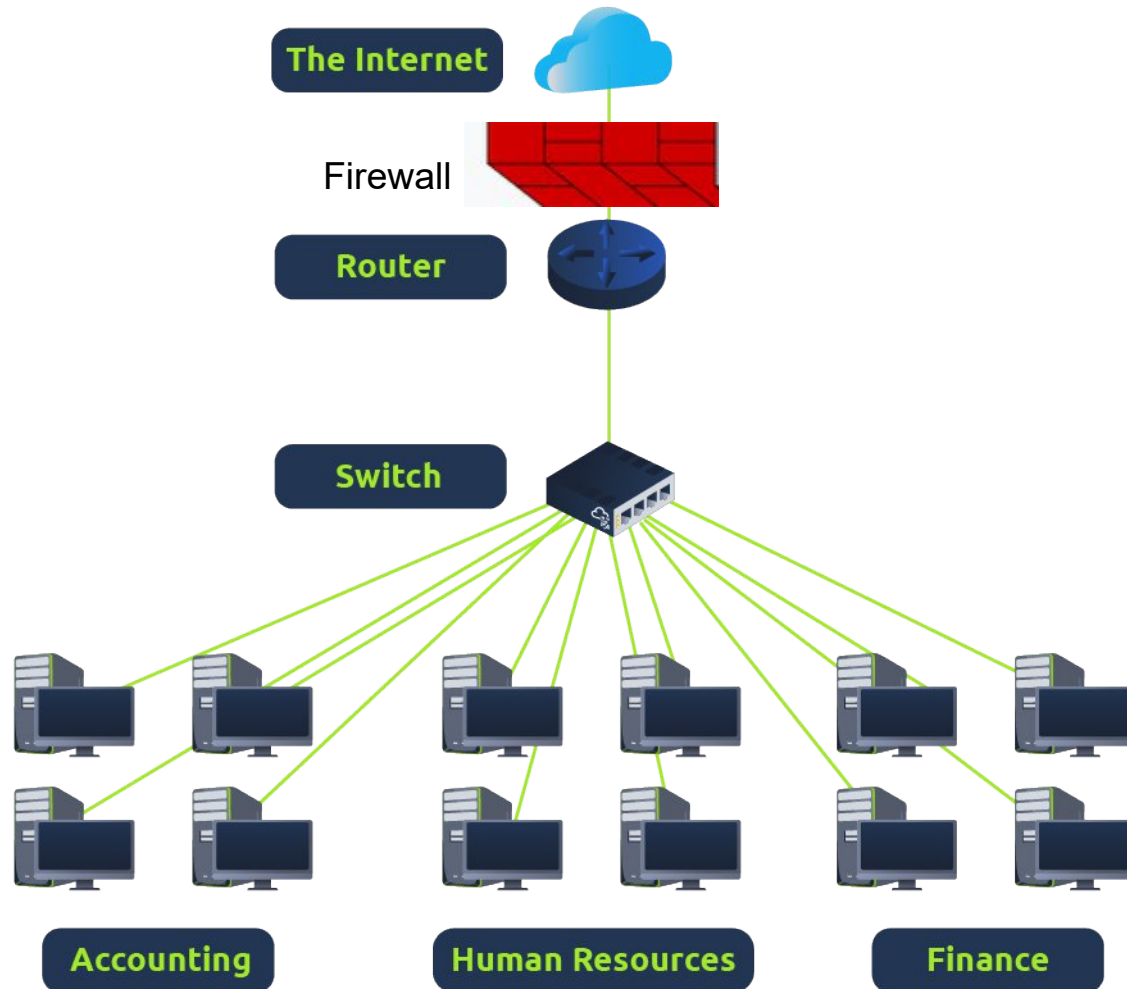


Switch

- ermöglicht die gemeinsame Nutzung von Ressourcen, indem alle Geräte, z.B. Computer, Drucker und Server, in einem Netzwerk miteinander verbunden werden.
- Die verbundenen Geräte können über den Switch Informationen teilen und miteinander kommunizieren, unabhängig davon, wo in einem Gebäude oder wo auf einem Campus sie sich gerade befinden.

Router

- verbindet mehrere Switches und die entsprechenden Netzwerke miteinander, um ein noch grösseres Netzwerk zu bilden.
- Der Router verbindet nicht nur mehrere Netzwerke miteinander, sondern ermöglicht Netzwerkgeräten und mehreren Benutzern zudem den Zugriff auf das Internet.
- dient als Dispatcher, der Datenverkehr weiterleitet und die effizienteste Route auswählt, über die Informationen in Form von Datenpaketen in einem Netzwerk übertragen werden.

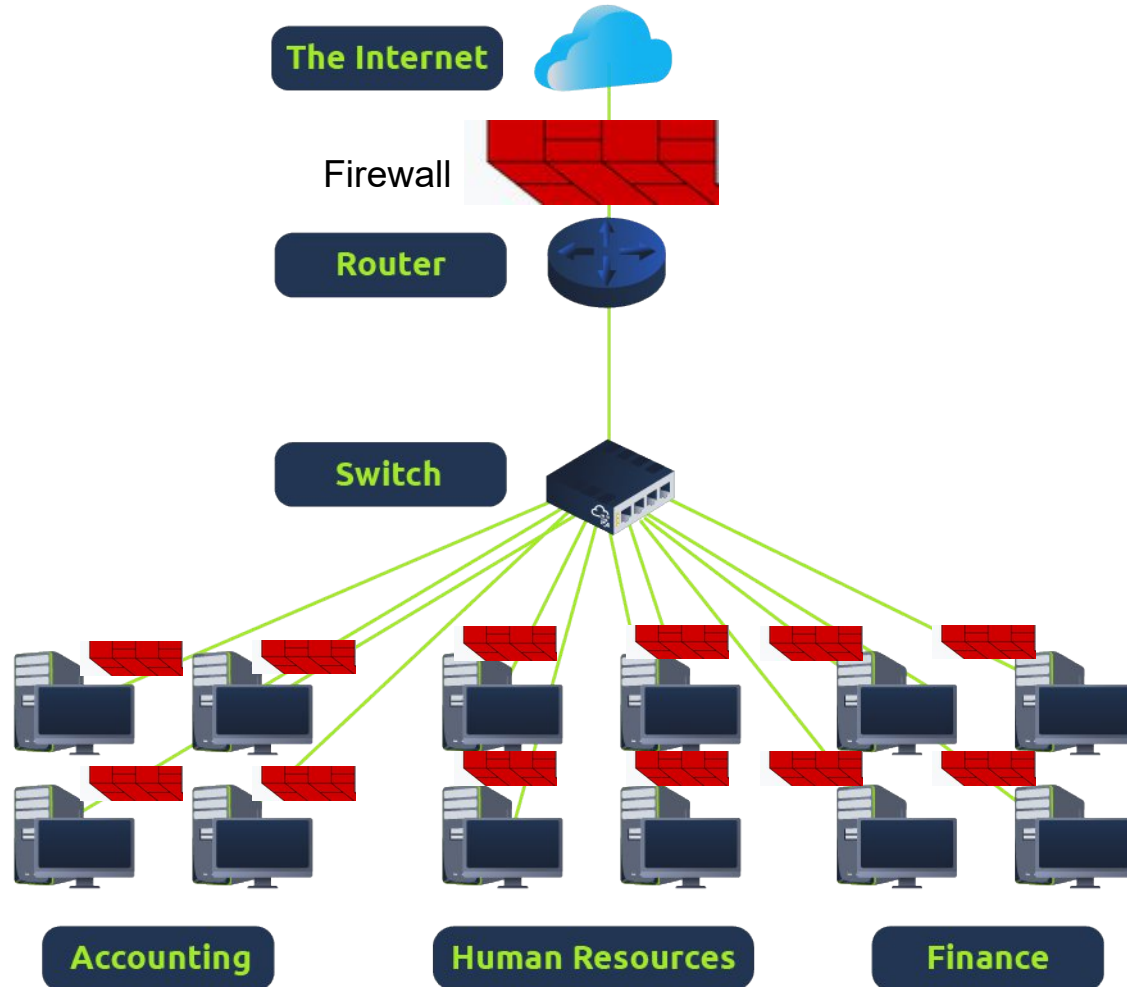


Firewall (extern)

- kontrolliert die Verbindung zwischen zwei Netzen
- dient dazu, den Netzwerkzugriff zu beschränken, basierend auf Absender- oder Zieladresse und genutzten Diensten.
- überwacht den durchlaufenden Datenverkehr
- entscheidet anhand festgelegter Regeln, ob bestimmte Netzwerkpakete durchgelassen werden oder nicht.

https://www.cisco.com/c/de_ch/solutions/small-business/resource-center/networking/network-switch-vs-router.html#~switches

<https://tryhackme.com/room/introtolan>



Firewall (extern)

- kontrolliert die Verbindung zwischen zwei Netzen
- dient dazu, den Netzwerkzugriff zu beschränken, basierend auf Absender- oder Zieladresse und genutzten Diensten.
- überwacht den durchlaufenden Datenverkehr
- entscheidet anhand festgelegter Regeln, ob bestimmte Netzwerkpakete durchgelassen werden oder nicht.

Firewall (personal)

- filtert den ein- und ausgehenden Datenverkehr eines PCs auf dem Rechner selbst

https://www.cisco.com/c/de_ch/solutions/small-business/resource-center/networking/network-switch-vs-router.html#~switches
<https://tryhackme.com/room/introtolan>

Internetkommunikation: ISO/OSI-Referenzmodell



OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA TLS, SOCKS
Darstellung (6)		
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>



IP-Adresse

- Adresse in Computernetzen, die auf dem **Internetprotokoll (IP)** basieren
- Geräte somit adressierbar und erreichbar.
- IP-Adresse ähnlich der Postanschrift auf einem Briefumschlag:
 - Datenpakete erhalten einer IP-Adresse, die den Empfänger eindeutig identifiziert. Aufgrund dieser Adresse können die Router, entscheiden, in welche Richtung das Paket weitertransportiert werden soll.

There's no place like
127.0.0.1



There's no place like
127.0.0.1

IP-Adresse

- Adresse in Computernetzen, die auf dem **Internetprotokoll (IP)** basieren
- Geräte somit adressierbar und erreichbar.
- IP-Adresse ähnlich der Postanschrift auf einem Briefumschlag:
 - Datenpakete erhalten einer IP-Adresse, die den Empfänger eindeutig identifiziert. Aufgrund dieser Adresse können die Router, entscheiden, in welche Richtung das Paket weitertransportiert werden soll.
- **IPv4**-Adressen (Version 4) besteht aus vier Bytes welche mit einem Punkt getrennt werden, beispielsweise 192.0.2.42.
- Technisch gesehen ist die Adresse eine 32-stellige (IPv4) Binärzahl

There's no place like
127.0.0.1

IP-Adresse

- Adresse in Computernetzen, die auf dem **Internetprotokoll (IP)** basieren
- Geräte somit adressierbar und erreichbar.
- IP-Adresse ähnlich der Postanschrift auf einem Briefumschlag:
 - Datenpakete erhalten einer IP-Adresse, die den Empfänger eindeutig identifiziert. Aufgrund dieser Adresse können die Router, entscheiden, in welche Richtung das Paket weitertransportiert werden soll.
- **IPv4**-Adressen (Version 4) besteht aus vier Bytes welche mit einem Punkt getrennt werden, beispielsweise 192.0.2.42.
- Technisch gesehen ist die Adresse eine 32-stellige (IPv4) Binärzahl
- **IPv6**-Adressen besteht aus acht Blöcke zu jeweils 16 Bit (4 Hexadezimalstellen) z.B. 2001:0db8:85a3:08d3:0000:8a2e:7f00:0001
- Führende Nullen dürfen beim Schreiben weggelassen werden:
2001:db8:85a3:8d3:0000:8a2e:7f00:1
- Aufeinander folgende Nullblöcke durch :: gekennzeichnet werden
2001:db8:85a3:8d3::8a2e:7f00:1
- die letzten beide Blöcke dürfen wie IPv4 geschrieben werden (für Einbettung IPv4 in IPv6)
2001:db8:85a3:8d3::8a2e:127.0.0.1
- ist eine 128-stellige (IPv6) Binärzahl.
- IPv6 wird IPv4 vollständig ablösen, da es deutlich mehr Adressen bietet
- ggf. Zurückdrängen der Anonymität im Internet

<https://de.wikipedia.org/wiki/IP-Adresse>

<https://de.wikipedia.org/wiki/IPv6>

Portnummern



Ports werden von Transportprotokollen wie TCP oder UDP verwendet, um empfangene Daten einer Anwendung zuzuordnen. In vielen Anwendungen und Netzwerkdiensten ist dafür eine feste Portnummer vorgesehen.



Systemports (0–1023)

Der Bereich 0 bis 1023 umfasst die Systemports, auch als wohlbekannte Ports bezeichnet.

Ports werden von Transportprotokollen wie TCP oder UDP verwendet, um empfangene Daten einer Anwendung zuzuordnen. In vielen Anwendungen und Netzwerkdiensten ist dafür eine feste Portnummer vorgesehen.

67	—	UDP	Bootstrap Protocol (BOOTP) Server; auch genutzt von DHCP
68	—	UDP	Bootstrap Protocol (BOOTP) Client; auch genutzt von DHCP
69	—	UDP	Trivial File Transfer Protocol (TFTP)
70	TCP	—	Gopher-Protokoll
71	TCP	—	Genius-Protokoll
79	TCP	—	Finger
80	TCP	UDP	Hypertext Transfer Protocol (HTTP)
83	TCP	—	MIT ML Device
88	TCP	UDP	Kerberos-Authentifizierungssystem
90	TCP	UDP	dnsix (DoD Network Security for Information Exchange) Securit Attribute Token Map





Systemports (0–1023)

Der Bereich 0 bis 1023 umfasst die Systemports, auch als wohlbekannte Ports bezeichnet.

Registrierte Ports (1024–49151)

Diese Ports von Nr. 1024 bis Nr. 49151 (0400_{hex} bis BFFF_{hex}) sind *registrierte Ports*. Auf den meisten Systemen können diese Ports von normalen Nutzern ohne besondere Rechte verwendet werden.

Ports werden von Transportprotokollen wie TCP oder UDP verwendet, um empfangene Daten einer Anwendung zuzuordnen. In vielen Anwendungen und Netzwerkdiensten ist dafür eine feste Portnummer vorgesehen.

25565	TCP	—	Minecraft Ausgewiesener Server
25888	—	UDP	Xfire (Firewall Report, UDP_IN)
25999	TCP	—	Xfire
26000	—	UDP	Quake von id Software
26000	TCP	—	Quake von id Software
26000	TCP	—	EVE Online MMORPG von CCP
26900	TCP	—	EVE Online MMORPG von CCP
26901	TCP	—	EVE Online MMORPG von CCP



Portnummern



Systemports (0–1023)

Der Bereich 0 bis 1023 umfasst die Systemports, auch als well-known ports bezeichnet, um empfangene Nachrichten zuzuordnen. In diesen Ports sind viele Netzwerkdiens

transportprotokollen wie
endet, um empfangene
ung zuzuordnen. In
n und Netzwerkdiensten
ortnummer vorgesehen.

Registrierte Ports (1024–49151)

Diese Ports von Nr. 1024 bis Nr. 49151 (0400_{hex} bis BFFF_{hex}) sind für die Registrierung von normalen Nutzern ohne besondere Rechte vorgesehen.

```
22  #include "quakedef.h"
23  #include "net_vcr.h"
24
25  qsocket_t *net_activeSockets = NULL;
26  qsocket_t *net_freeSockets = NULL;
27  int net_numsockets = 0;
28
29  qboolean serialAvailable = false;
30  qboolean ipxAvailable = false;
31  qboolean tcpipAvailable = false;
32
33  int net_hostport;
34  int DEFAULTnet_hostport = 26000;
```

25565	TCP		
25888	—	UD	
25999	TCP	—	Xfire
26000	—	UDP	Quake von id Software
26000	TCP	—	Quake von id Software
26000	TCP	—	EVE Online MMORPG
26900	TCP	—	EVE Online MMORPG
26901	TCP	—	EVE Online MMORPG



Internetkommunikation: ISO/OSI-Referenzmodell



OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA
Darstellung (6)		
Sitzung (5)		
Transport (4)	Transport	TLS, SOCKS
Vermittlung (3)	Internet	TCP, UDP, SCTP
Sicherung (2)	Netzzugang	IP (IPv4, IPv6), ICMP (über IP)
Bitübertragung (1)		
		Ethernet, Token Bus, Token Ring, FDDI

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

Netzwerkprotokoll: TCP



TCP Segment Header Format

Bit #	0	7	8	15	16	23	24	31
0	Source Port				Destination Port			
32	Sequence Number							
64	Acknowledgment Number							
96	Data Offset	Res	Flags		Window Size			
128	Header and Data Checksum				Urgent Pointer			
160...	Options							

„TCP [is] like making a reliable phone call, ensuring that all information is delivered correctly.“ [2]

Das **Transmission Control Protocol** (TCP) [1]

- ist ein Netzwerkprotokoll (d.h. definiert wie Daten zwischen Netzwerkkomponenten ausgetauscht werden)
- zuverlässig, verbindungsorientiert, paketvermittelt
- entwickelt von Kahn und Cerf an 1973
- erste Standardisierung von TCP 1981 als RFC 793 (später RFC 9293)
- stellt eine Verbindung zwischen zwei Endpunkten einer Netzverbindung (Sockets) her.
- Auf dieser Verbindung können in beide Richtungen Daten übertragen werden
- setzt in den meisten Fällen auf das IP (Internet-Protokoll) auf, daher häufig auch als „TCP/IP-Protokoll“ bezeichnet.
- TCP lange Zeit als fast ausschließliches Transportprotokoll für das WWW, E-Mail und viele andere populäre Netzdienste (seit 2021 verschlüsselte Transportprotokoll QUIC standardisiert)
- positive Eigenschaften:
 - Datenverluste werden erkannt und automatisch behoben
 - Datenübertragung ist in beiden Richtungen möglich
 - Netzüberlastung wird verhindert
 - und weitere

Netzwerkprotokoll: UDP



„UDP [is] more like sending letters. They’re faster but might get lost.“ [2]

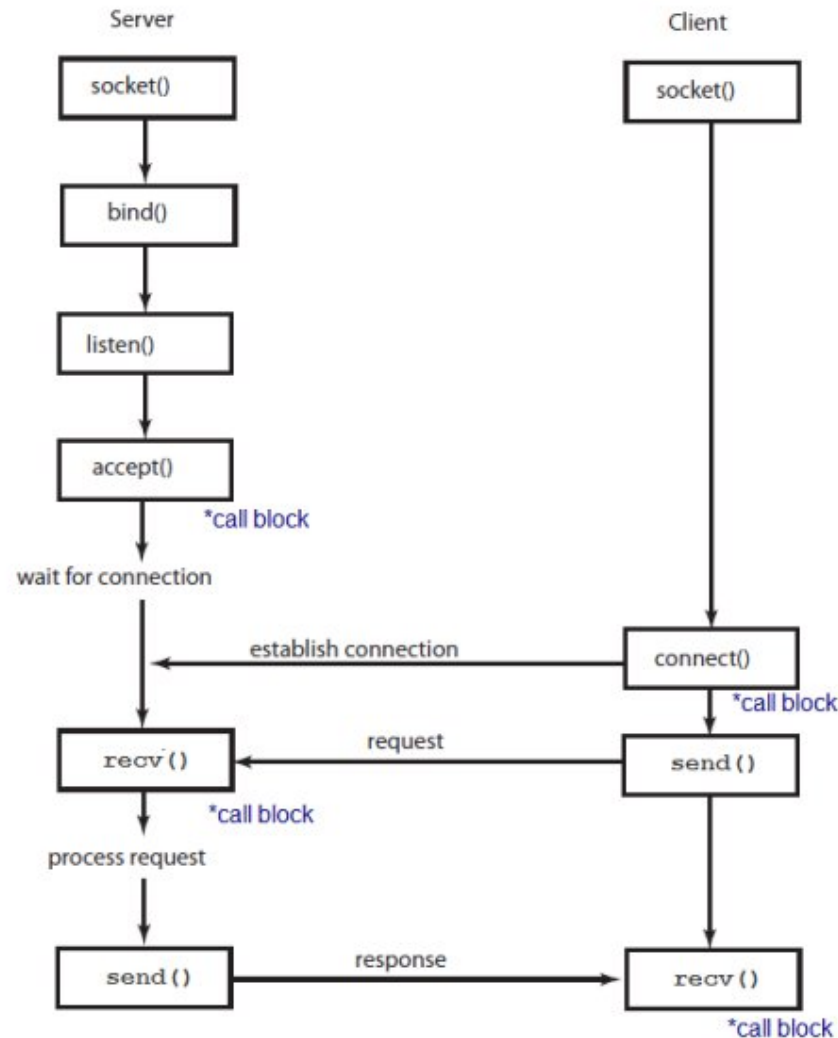
UDP Datagram Header Format

Bit #	0	7	8	15	16	23	24	31
0	Source Port				Destination Port			
32	Length				Header and Data Checksum			

Das **User Datagram Protocol** (UDP) [1]

- minimales, verbindungsloses Netzwerkprotokoll
 - in IP-basierten Rechnernetzen
 - 1980 als Internetstandard veröffentlicht
 - Alternative zu TCP
 - für Anwendungen eingeführt, die keine zuverlässige Datenübertragung benötigen
-
- aktuelle Anwendungszwecke:
 - Onlinespiele
 - IP-Telefonie
 - Video Streamingda geringe Verzögerungszeit wichtiger als eine zuverlässige Übertragung

TCP Kommunikation (Ablaufdiagramm)



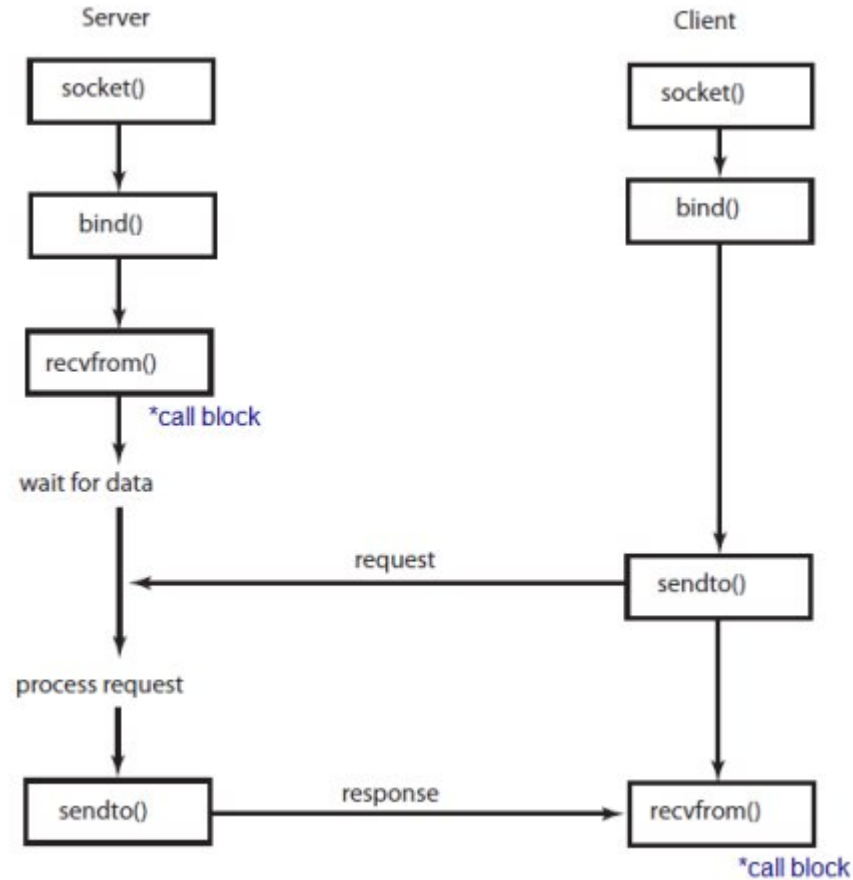
Server

1. Erzeugen eines **sockets** (file descriptor)
2. An eine Adresse binden (**bind**). Festlegen eines Ports.
3. Am Port hören (**listen**) und darauf warten, dass eine Verbindung erstellt wird
4. Verbindung vom Client akzeptieren (**accept**)
5. Schreiben und Lesen (**send/recv**) - wie beim Schreiben in eine Datei
6. Beenden von Schreiben/Lesen (**shutdown**)
7. Aufräumen (**close**)

Client

1. Erzeugen eines **sockets** (file descriptor)
2. Mit dem Server verbinden (**connect**)
3. Schreiben und Lesen (**send/recv**) - wie beim Schreiben in eine Datei
4. Beenden von Schreiben/Lesen (**shutdown**)
5. Aufräumen (**close**)

UDP Kommunikation (Ablaufdiagramm)



Bei UDP ist der Verbindungsaufbau im Ablaufdiagramm entsprechend einfacher als bei TCP.

Internetkommunikation: ISO/OSI-Referenzmodell

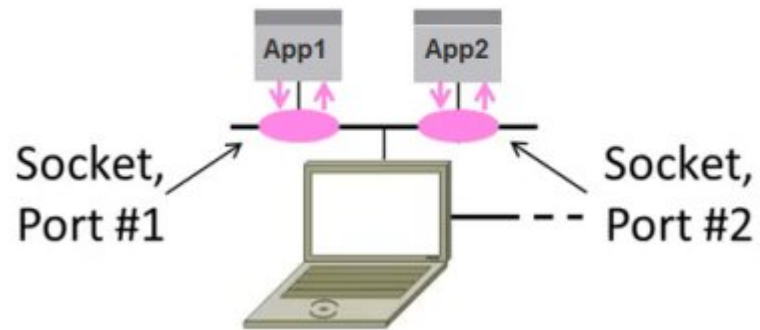


OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA TLS, SOCKS
Darstellung (6)		
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

Sockets let apps attach to the local network at different **ports**



Sockets

- (von engl. Sockel, Steckverbindung oder Steckdose)
- Endpunkt einer Netzverbindung
- plattformunabhängige standardisierte Schnittstelle (API) zwischen der Netzwerkprotokoll-Implementierung des Betriebssystems und der eigentlichen Anwendungssoftware.
- Ein Computerprogramm fordert einen Socket vom Betriebssystem an.
- Das Betriebssystem hat die Aufgabe, alle benutzten Sockets sowie die zugehörigen Verbindungsinformationen zu verwalten.

Internetkommunikation: ISO/OSI-Referenzmodell



OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA
Darstellung (6)		TLS, SOCKS
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

HTTP

- Hypertext Transfer Protocol
- developed by Tim Berners-Lee and his team between 1989-1991.
- HTTP is the set of rules used for communicating with web servers for the transmitting of webpage data, whether that is HTML, Images, Videos, etc.

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the p [Questions](#) .

[What's out there?](#)

Pointers to the world's online information, [subjects](#) , [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#) , [X11](#) , [Viola](#) , [NeXTStep](#) , [Servers](#) , [Tools](#) , [Mail](#)

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#) , etc.

**Tim Berners-Lee launched the world's first website,
World Wide Web, on August 6th, 1991**

HTTP

- Hypertext Transfer Protocol
- developed by Tim Berners-Lee and his team between 1989-1991.
- HTTP is the set of rules used for communicating with web servers for the transmitting of webpage data, whether that is HTML, Images, Videos, etc.

HTTPS

- HyperText Transfer Protocol Secure
- secure version of HTTP
- HTTPS data is encrypted
- data encrypted
- but also assurances that you're talking to the correct web server

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#), [Policy](#), [Questions](#).

[What's out there?](#)

Pointers to the world's online information, [subjects](#), [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#), [X11 Viola](#), [NeXTStep](#), [Servers](#), [Tools](#), [Mail robot](#), [Library](#).)

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

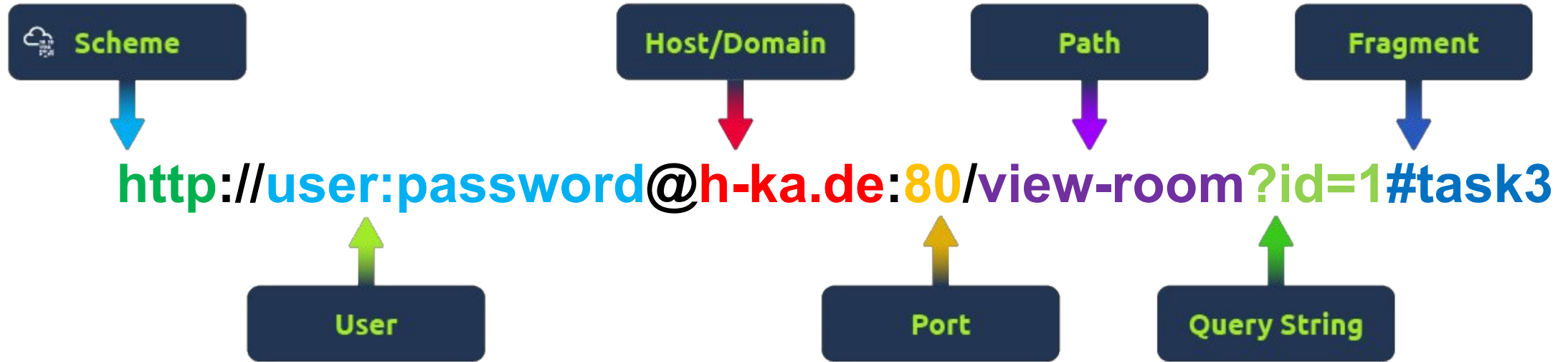
If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#), etc.

**Tim Berners-Lee launched the world's first website,
World Wide Web, on August 6th, 1991**

HTTP Protokoll



Scheme: This instructs on what protocol to use for accessing the resource such as , HTTPS, FTP (File Transfer Protocol).

User: Some services require authentication to log in, you can put a username and password into the URL to log in.

Host: The domain name or IP address of the server you wish to access.

Port: The Port that you are going to connect to, usually 80 for and 443 for HTTPS, but this can be hosted on any port between 1 - 65535.

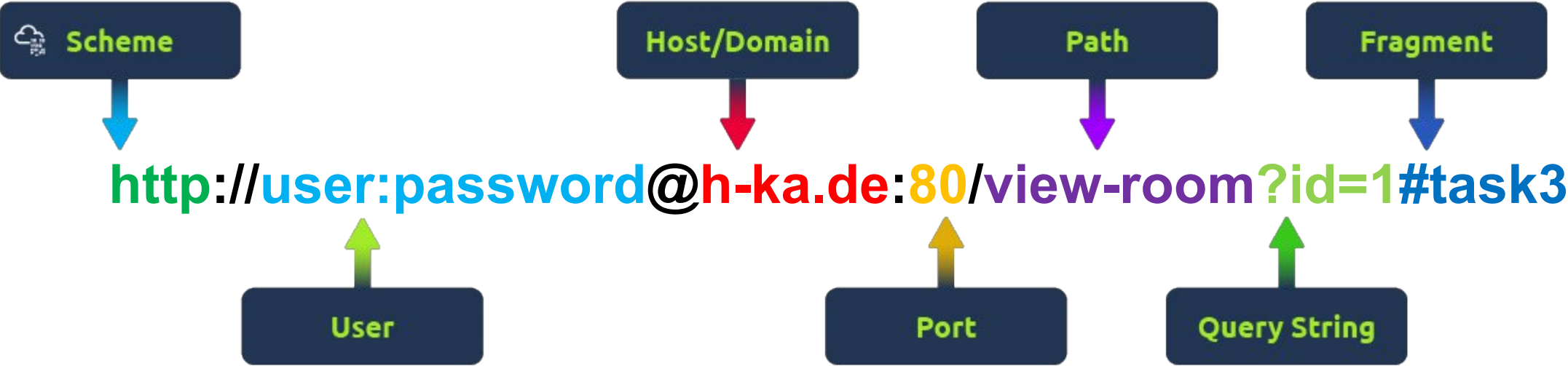
Path: The file name or location of the resource you are trying to access.

Query String: Extra bits of information that can be sent to the requested path. For example, `/blog?id=1` would tell the blog path that you wish to receive the blog article with the id of 1.

Fragment: This is a reference to a location on the actual page requested. This is commonly used for pages with long content and can have a certain part of the page directly linked to it, so it is viewable to the user as soon as they access the page.

<https://tryhackme.com/room/httpindetail>

HTTP Protokoll / Domain Name System (DNS)



„Das Domain Name System (DNS) ist das Telefonbuch des Internets. Wenn Benutzer Domainnamen wie „google.com“ oder „nytimes.com“ in ihren Webbrowser eingeben, ist das DNS dafür verantwortlich, die richtige IP-Adresse für diese Websites zu finden.“

<https://www.cloudflare.com/de-de/learning/dns/what-is-a-dns-server/>

DNS-Server

Anbieter	Sitz	Erste IPv4	Alternative IPv4
<u>Cloudflare</u>	USA	1.1.1.1	1.0.0.1
<u>Google</u>	USA	8.8.8.8	8.8.4.4
<u>Quad9</u>	Schweiz	9.9.9.9	149.112.112.112
<u>DNS.WATCH</u>	Deutschland	84.200.69.80	84.200.70.40

HTTP Example: Connecting to the first Webpage



Request:

WHAT

Response:

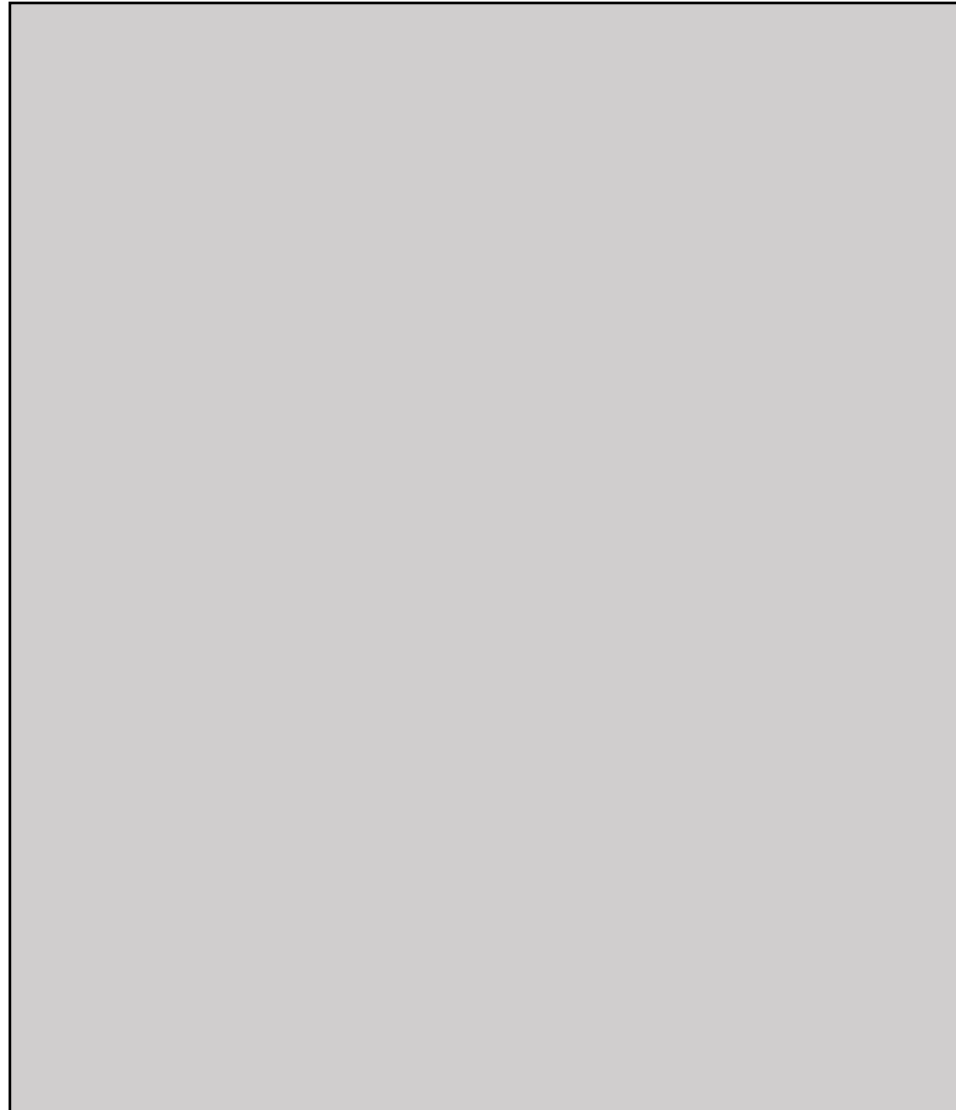
HTTP Example: Connecting to the first Webpage



Request:

```
GET /hypertext/WWW/TheProject.html HTTP/1.1  
Host: info.cern.ch
```

Response:



HTTP Example: Connecting to the first Webpage



Request:

```
GET /hypertext/WWW/TheProject.html HTTP/1.1
Host: info.cern.ch
```

Response:

```
HTTP/1.1 200 OK
Date: Mon, 25 Mar 2024 08:52:55 GMT
Server: Apache
Last-Modified: Thu, 03 Dec 1992 08:37:20 GMT
ETag: "8a9-291e721905000"
Accept-Ranges: bytes
Content-Length: 2217
Connection: close
Content-Type: text/html

<HEADER>
<TITLE>The World Wide Web project</TITLE>
<NEXTID N="55">
</HEADER>
<BODY>
<H1>World Wide Web</H1>The WorldWideWeb (W3) is a wide-area<A
NAME=0 HREF="WhatIs.html">
hypermedia</A> information retrieval
initiative aiming to give universal
access to a large universe of documents.<P>
Everything there is online about
W3 is linked directly or indirectly
to this document, including an <A
NAME=24 HREF="Summary.html">executive
summary</A> of the project, <A
NAME=29 HREF="Administration/Mailing/Overview.html">Mailing lists</A>
, <A
NAME=30 HREF="Policy.html">Policy</A> , November's <A
NAME=34 HREF="News/9211.html">W3 news</A> ,
<A
NAME=41 HREF="FAQ/List.html">Frequently Asked Questions</A> .
<DL>
<DT><A
NAME=44 HREF=" /DataSources/Top.html">What's out there?</A>
```

HTTP Request & Response



Example Request:

```
GET / HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Referer: https://tryhackme.com/
```

- Line 1:** This request is sending the GET method, request the home page with / and telling the web server we are using HTTP protocol version 1.1.
- Line 2:** We tell the web server we want the website tryhackme.com
- Line 3:** We tell the web server we are using the Firefox version 87 Browser
- Line 4:** We are telling the web server that the web page that referred us to this one is
- Line 5:** requests always end with a blank line to inform the web server that the request has finished.

Example Response:

```
HTTP/1.1 200 OK
Server: nginx/1.15.8
Date: Fri, 09 Apr 2021 13:34:03 GMT Content-
Type: text/html
Content-Length: 98

<html>
<head>
  <title>TryHackMe</title>
</head>
<body>
Welcome To TryHackMe.com
</body></html>
```

- Line 1:** 1.1 is the version of the HTTP protocol the server is using and then followed by the HTTP Status Code in this case "200 Ok" which tells us the request has completed successfully.
- Line 2:** This tells us the web server software and version number.
- Line 3:** The current date, time and timezone of the web server.
- Line 4:** The Content-Type header tells the client what sort of information is going to be sent, such as HTML, images, videos, pdf, .
- Line 5:** Content-Length tells the client how long the response is, this way we can confirm no data is missing.
- Line 6:** response contains a blank line to confirm the end of the HTTP response
- Lines 7-14:** The information that has been requested, in this instance the homepage.

HTTP Headers & Methods



Common Request Headers

Host: Some web servers host multiple websites so by providing the host headers you can tell it which one you require, otherwise you'll just receive the default website for the server.

User-Agent: This is your browser software and version number, telling the web server your browser software helps it format the website properly for your browser and also some elements of HTML, JavaScript and CSS are only available in certain browsers.

Content-Length: When sending data to a web server such as in a form, the content length tells the web server how much data to expect in the web request. This way the server can ensure it isn't missing any data.

Accept-Encoding: Tells the web server what types of compression methods the browser supports so the data can be made smaller for transmitting over the internet.

Cookie: Data sent to the server to help remember your information (see cookies task for more information).

GET Request

This is used for getting information from a web server.

POST Request

This is used for submitting data to the web server and potentially creating new records

PUT Request

This is used for submitting data to a web server to update information

DELETE Request

This is used for deleting information/records from a web server.

Common Response Headers

Set-Cookie: Information to store which gets sent back to the web server on each request (see cookies task for more information).

Cache-Control: How long to store the content of the response in the browser's cache before it requests it again.

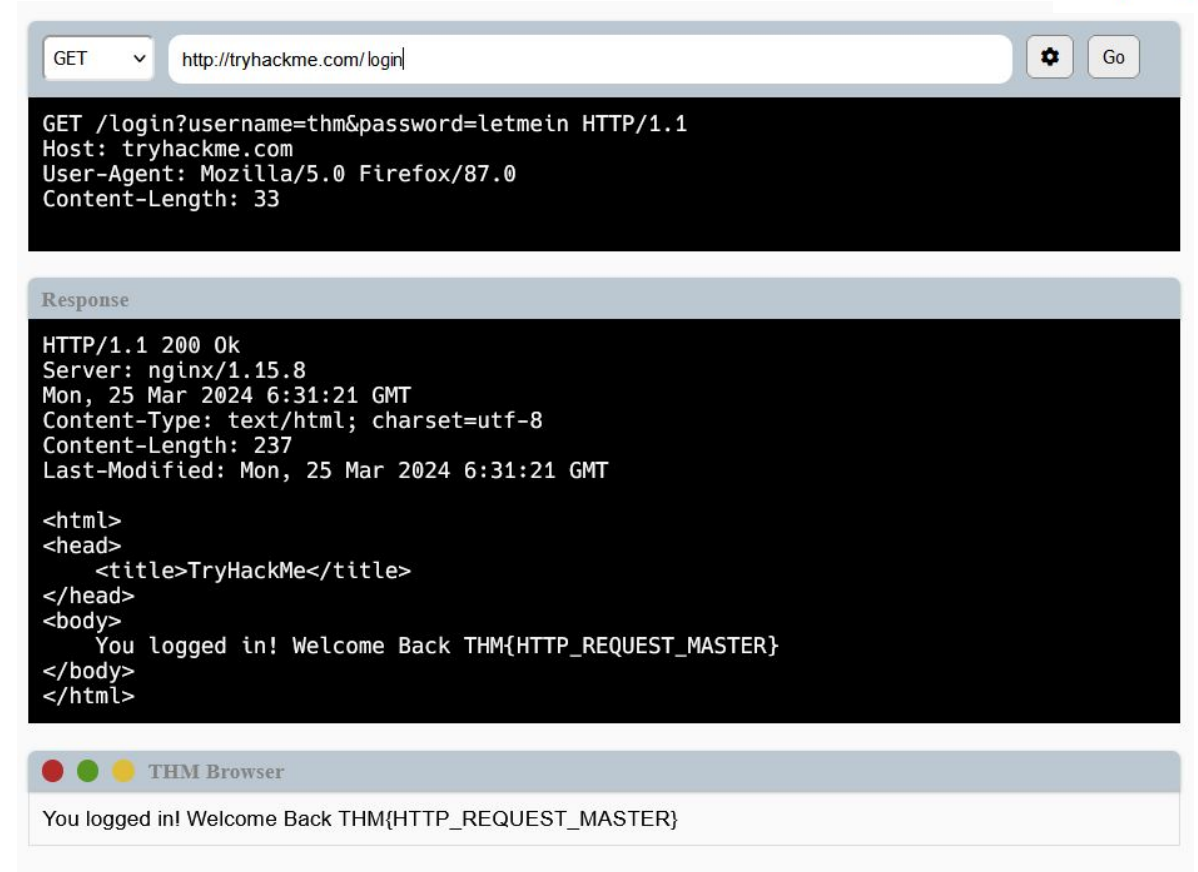
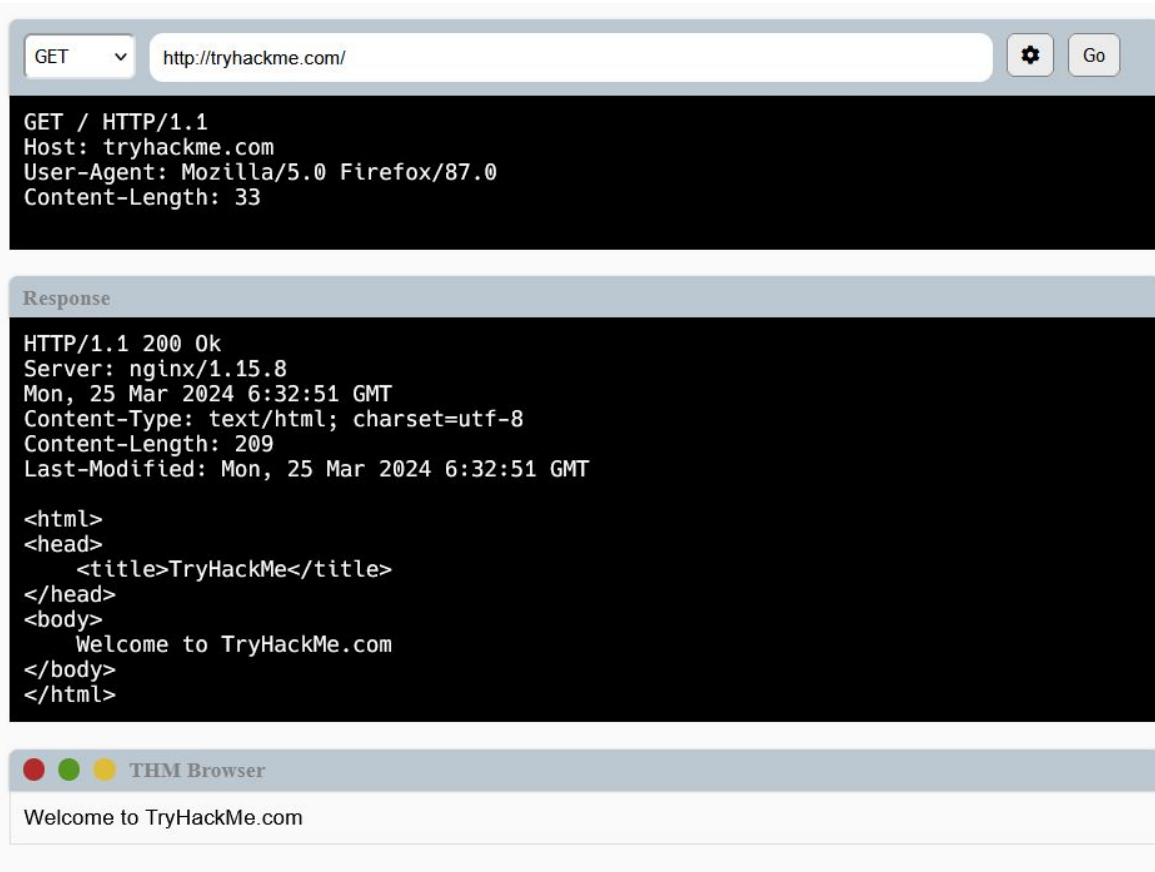
Content-Type: This tells the client what type of data is being returned, i.e., HTML, CSS, JavaScript, Images, PDF, Video, etc. Using the content-type header the browser then knows how to process the data.

Content-Encoding: What method has been used to compress the data to make it smaller when sending it over the internet.

<https://tryhackme.com/room/httpindetail>
<https://www.jmarshall.com/easy/http/>



Examples GET



GET Request

This is used for getting information from a web server.

POST Request

This is used for submitting data to the web server and potentially creating new records

PUT Request

This is used for submitting data to a web server to update information

DELETE Request

This is used for deleting information/records from a web server.



Examples POST/PUT



POST /login HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 33

username=thm&password=letmein

Response

HTTP/1.1 200 Ok
Server: nginx/1.15.8
Mon, 25 Mar 2024 6:31:21 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 237
Last-Modified: Mon, 25 Mar 2024 6:31:21 GMT

<html>
<head>
 <title>TryHackMe</title>
</head>
<body>
 You logged in! Welcome Back THM{HTTP_REQUEST_MASTER}
</body>
</html>

THM Browser

You logged in! Welcome Back THM{HTTP_REQUEST_MASTER}

PUT /user/2 HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 14

username=admin

Response

HTTP/1.1 200 Ok
Server: nginx/1.15.8
Mon, 25 Mar 2024 6:29:1 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 232
Last-Modified: Mon, 25 Mar 2024 6:29:1 GMT

<html>
<head>
 <title>TryHackMe</title>
</head>
<body>
 Username changed to admin THM{USER_HAS_UPDATED}
</body>
</html>

THM Browser

Username changed to admin THM{USER_HAS_UPDATED}

GET Request

This is used for getting information from a web server.

POST Request

This is used for submitting data to the web server and potentially creating new records

PUT Request

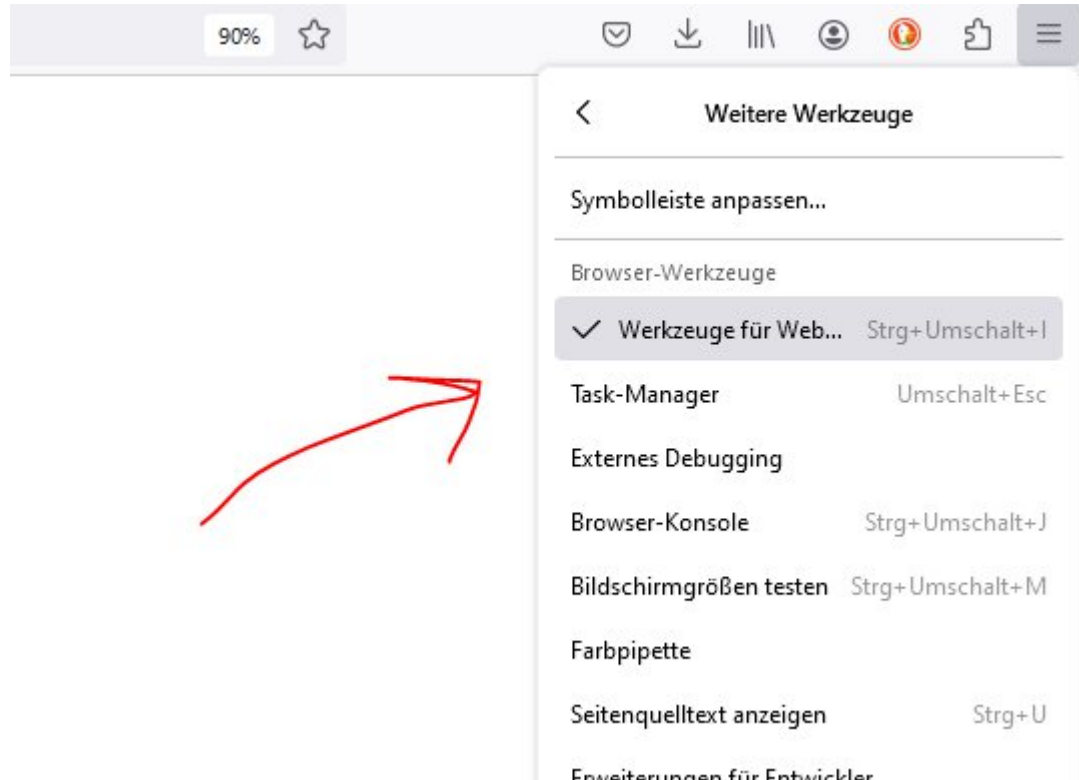
This is used for submitting data to a web server to update information



HTTP Example: Cookie Setting



HTTP Interaktion: Ansehen



HTTP Interaktion: Ansehen



Über Google Store

Gmail Bilder Anmelden



Google Suche Auf gut Glück!

Deutschland

Werbeprogramme Unternehmen Wie funktioniert die Google Suche? Drei Jahrzehnte Klimaschutz: Jede Entscheidung zählt Datenschutzerklärung Nutzungsbedingungen Einstellungen

Adressen durchsuchen

Status	Methode	Host	Datei	Initiator	Typ	Übertragen	Größe
200	GET	www.google.com	/	document	html	58,09 kB	201,77 kB
200	GET	www.google.com	m=cdo	script	js	Aus Cache	863,82 kB
200	GET	www.google.com	googlelogo_color_272x92dp.png	imageset	png	Aus Cache	5,97 kB
200	GET	www.gstatic.com	rs=AA2WtUqK5zupohdywxEvdO08ckxPTTKAA	107 (script)	js	Aus Cache	216,85 kB
200	GET	www.gstatic.com	rs=AA2WtUES-lZesaZSW0VjqDmlw0EEOG6Fw	107 (stylesheet)	css	Aus Cache	1,66 kB
200	GET	www.google.com	desktop_searchbox_sprites318_hr.webp	6 (img)	webp	Aus Cache	660 B
204	POST	www.google.com	gen_204?s=webhp&t=ft&atyp=csi&ei=_zgBZsyuCMuL7NYPkveD0Ao&rt=wsrt.159,af	11 (beacon)	html	653 B	0 B
200	GET	apis.google.com	cb=gapi.loaded_0	rs=AA2WtUqK5zupohdywxEvd...	js	Aus Cache	0 B
204	POST	www.google.com	gen_204?atyp=csi&ei=_zgBZsyuCMuL7NYPkveD0Ao&s=webhp&t=all&wh=610&imn=1	m=cdo	html	653 B	0 B
200	GET	www.google.com	search?q&cp=0&client=gws-wiz&xssi=t&gs_l=de&authuser=0&psi=_zgBZsy	m=cdo	json	3,01 kB	3,33 kB
200	GET	www.google.com	m=B2qIPe,DhPIme,GU4Gab,MpJwZc,NzU6V,UUqVe,Wo3n8,aa,abd,async,epYOx,ifl,ms4i	m=cdo	js	Aus Cache	488,11 kB
204	GET	www.google.com	client_204?atyp=i&biw=2144&bih=610&dpr=0.8955223880597015&ei=_zgBZsyuCMuL	3 (img)	html	694 B	0 B
200	GET	www.google.com	rs=AA2WtUES-lZesaZSW0VjqDmlw0EEOG6Fw	m=cdo	js	Aus Cache	180,34 kB

Kopfzeilen Cookies Anfrage Antwort Zeit Sicherheit

GET

Scheme: https

Host: www.google.com

Filename: /

Adresse: [2a00:1450:4001:830::2004]:443

Status: 200

Version: HTTP/2

Übertragen: 58,09 kB (201,77 kB Größe)

Anfrage-Priorität: Highest

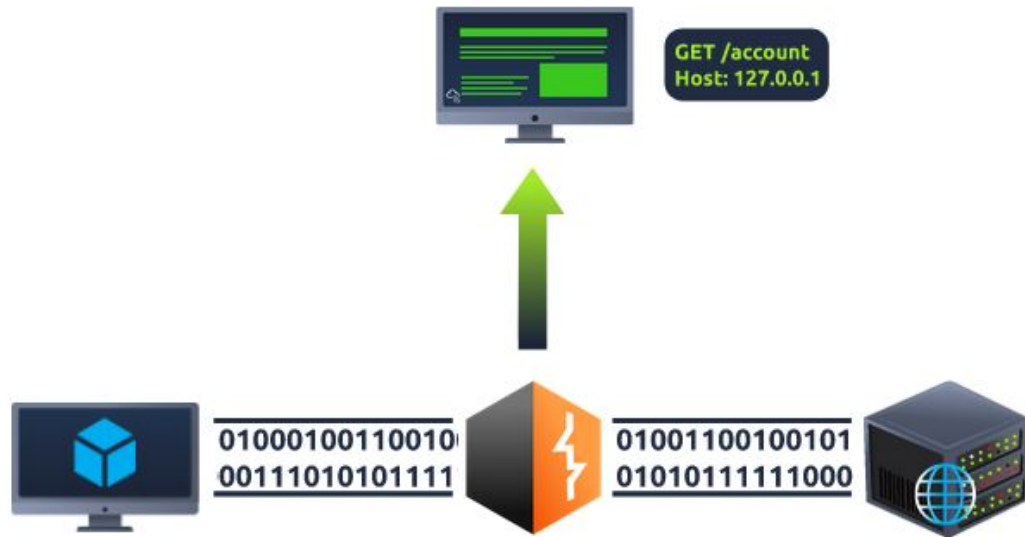
DNS-Auflösung: System

Antwortkopfzeilen (807 B)

Unformatiert



Burp Suite – Intercepting Communication



Burp Suite

- comprehensive solution for conducting web application penetration testing.
- industry standard tool for hands-on security assessments of web and mobile applications, including those that rely on application programming interfaces (APIs).
- captures and enables manipulation of all the HTTP/HTTPS traffic between a browser and a web server.
- ability to intercept, view, and modify web requests before they reach the target server or even manipulate responses before they are received by our browser makes Burp Suite an invaluable tool for manual web application testing.



C++



Winsock / Windows Sockets API (WSA)



```
#include <iostream>

#include <winsock2.h>

#pragma comment(lib, "Ws2_32.lib")
```

Include

winsock2.h

Load Lib

VS: #pragma comment(lib, "Ws2_32.lib")

g++ -o SocketServer SocketServer.cpp -lws2_32

- Winsock oder Windows Sockets ist eine Windows-Programmierschnittstelle (API) zum Zugriff auf Rechnernetze über Sockets.
- Winsock ergänzt Windows um die Internetprotokollfamilie und ist für die Verbindung des Computers mit dem Internet zuständig. Es wird von Programmen verwendet, um über das Internetprotokoll (IP) auf das Netzwerk zuzugreifen
- Winsock wurde 1992 von Microsoft entwickelt. Dazu wurden praktisch alle Funktionen von den Betriebssystemen Unix und BSD übernommen.
- Winsock 2.0 von 1994!



Winsock / Windows Sockets API (WSA)



```
#include <iostream>

#include <winsock2.h>

#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Start up Winsock
    WORD wVersionRequested;
    WSADATA wsaData; // WSADATA data structure
    wVersionRequested = MAKEWORD(2, 2);
    int err = WSAStartup(wVersionRequested, &wsaData);
    if (err != 0) {
        printf("WSAStartup failed with error: %d\n", err);
        return err;
    } else {
        printf("WSAStartup started!\n");
        WSACleanup(); // Clean up Winsock
        return 0;
    }
}
```

Initialize Winsock

In the server, the first step is to initialize the Winsock library. This involves loading the DLL and setting up the necessary variables.

see also:

<https://medium.com/@tharunappu2004/creating-a-simple-tcp-server-in-c-using-winsock-b75dde86dc39>
https://www.bogotobogo.com/cplusplus/sockets_server_client.php



Socket



```
#include <iostream>

#include <winsock2.h>

#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Initialize Winsock [...]
    // Initialize Socket
    SOCKET ConnectSocket = INVALID_SOCKET;
    ConnectSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (ConnectSocket == INVALID_SOCKET) {
        wprintf(L"socket failed with error: %ld\n", WSAGetLastError());
        WSACleanup();
    }
    return ConnectSocket;
}
```

Create a Socket

Next, we create a socket to handle communication.

The `socket()` function is used for this purpose.

It allows us to specify the address family, socket type, and protocol.

In our case, we are creating a TCP socket

AF_INET:	Internet Protocol version 4 (IPv4) address family
SOCK_STREAM:	This socket type uses the TCP
IPPROTO_TCP:	Using TCP

BTW:

AF_INET6 The Internet Protocol version 6 (IPv6) address family.

AF_IRDA The Infrared Data Association (IrDA) address family.

AF_BTH The Bluetooth address family.



Specify Connection (Client)



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Initialize Winsock [...]
    // Initialize Socket [...]
    // Specify Connection
    struct sockaddr_in clientService;
    inet_pton(AF_INET, "172.217.18.14", &clientService.sin_addr);
    clientService.sin_family = AF_INET;
    clientService.sin_port = htons(80);

    return 0;
}
```

Specify Connection

AF_INET: Internet Protocol version 4 (IPv4) address family
IP: 127.0.0.1 (Localhost)
Port: 80 (HTTP)

`inet_pton` wird benutzt um den String `"172.217.18.14"` in die korrekte 32bit Repräsentation für IPv4 zu konvertieren.



Connect to Server



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

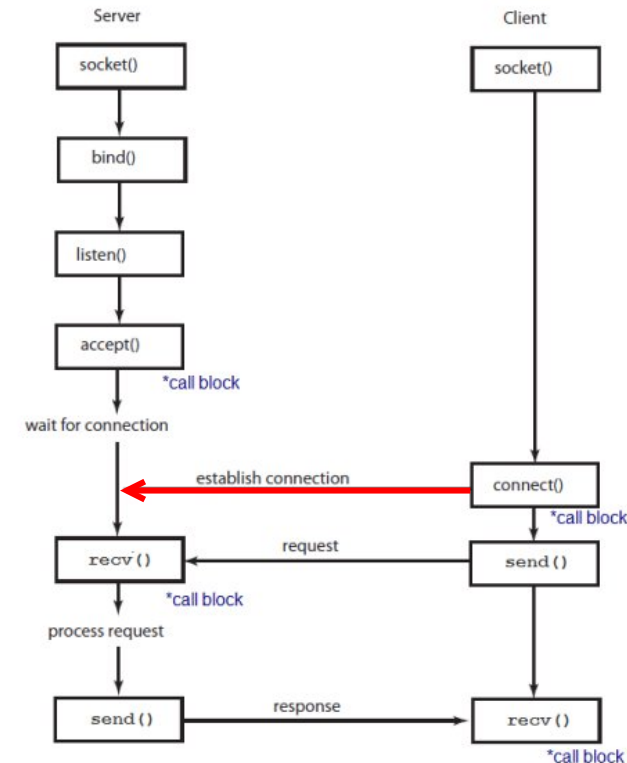
#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Initialize Winsock [..]
    // Initialize Socket [..]
    // Specify Connection [..]
    // Connect to Server
    int iResult;
    iResult = connect(ConnectSocket, (struct sockaddr*)&clientService, sizeof(clientService));
    if (iResult == SOCKET_ERROR) {
        wprintf(L"connect failed with error: %d\n", WSAGetLastError());
        // https://learn.microsoft.com/en-us/windows/win32/winsock/windows-sockets-error-codes-2
        closesocket(ConnectSocket);
        WSACleanup();
        return 1;
    } else {
        cout << "connected to Server \n"; return 0;
    }
}
```

Connect to the Server

Connect the client to the server using the connect() function.



Send



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

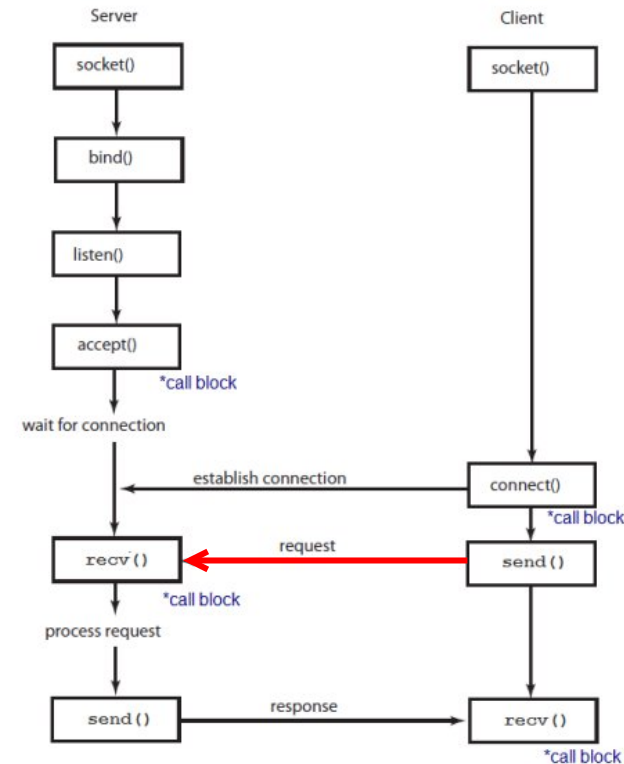
#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Initialize Winsock [..]
    // Initialize Socket [..]
    // Specify Connection [..]
    // Connect to Server [..]
    // Send Data to the Server
    const char* message = "Hello, server!";
    iResult = send(ConnectSocket, message, strlen(message), 0);
    if (iResult == SOCKET_ERROR) {
        wprintf(L"send failed with error: %d\n", WSAGetLastError());
        closesocket(ConnectSocket);
        WSACleanup();
        return 1;}
    printf("Bytes Sent: %d\n", iResult);
    return 0;
}
```

Send & Receive Data

Now that the client is set up and connected to a server, it can send and receive data.



Receive

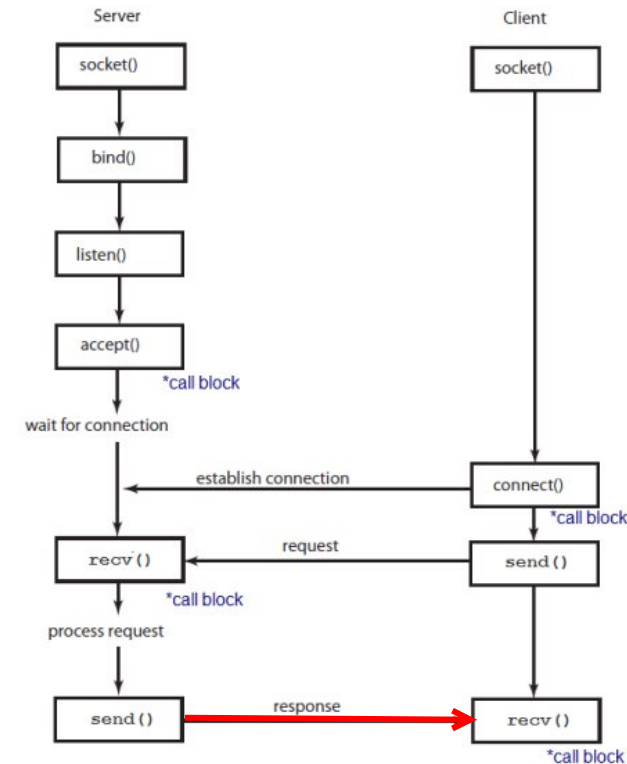


```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton
#pragma comment(lib, "Ws2_32.lib")
using namespace std;

int main() {
    // Initialize Winsock [..]
    // Initialize Socket [..]
    // Specify Connection [..]
    // Connect to Server [..]
    // Send Data to the Server [..]
    // Receive until the peer closes the connection
    int recvbuflen = DEFAULT_BUFLen;
    char recvbuf[DEFAULT_BUFLen] = "";
    do {iResult = recv(ConnectSocket, recvbuf, recvbuflen, 0);
        if (iResult > 0) {
            wprintf(L"Bytes received: %d\n", iResult);
            for (int jj = 0; jj < recvbuflen; jj++) {cout << recvbuf[jj];}
        } else if (iResult == 0) wprintf(L"Connection closed\n");
        else wprintf(L"recv failed with error: %d\n", WSAGetLastError());
    } while (iResult > 0);
    return 0;
}
```

Send & Receive Data

Now that the client is set up and connected to a server, it can send and receive data.



Close Socket und Clean up...



```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton
#pragma comment(lib, "Ws2_32.lib")
using namespace std;

int main() {
    // Initialize Winsock [..]
    // Initialize Socket [..]
    // Specify Connection [..]
    // Connect to Server [..]
    // Send Data to the Server [..]
    // Receive until the peer closes the connection [..]
    closesocket(ConnectSocket);
    WSACleanup(); // Clean up Winsock
    return 0;
}
```



C++ Implementation: HTTP Request



```
#include <iostream>
#include <string>
#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

#pragma comment(lib, "Ws2_32.lib")
#define DEFAULT_BUFLen 512

using namespace std;

int main() {
    // 1) Startup Winsock Application WSA
    WORD wVersionRequested;
    WSADATA wsaData; // WSADATA data structure
    wVersionRequested = MAKEWORD(2, 2);
    int err = WSAStartup(wVersionRequested, &wsaData);

    // 2) Initialize Socket
    SOCKET ConnectSocket = INVALID_SOCKET;
    ConnectSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    // 3) Specify Connection
    struct sockaddr_in clientService;
    inet_pton(AF_INET, "172.217.18.14", &clientService.sin_addr);
    clientService.sin_family = AF_INET;
    clientService.sin_port = htons(80);
```

```
    // 4) Connect to server.
    int iResult;
    iResult = connect(ConnectSocket, (SOCKADDR*)&clientService, sizeof(clientService));

    // 5) Send
    const char* rq_all = "GET / HTTP/1.1\r\nHost: www.google.de\r\nConnection: close\r\n\r\n";
    iResult = send(ConnectSocket, rq_all, (int)strlen(rq_all), 0);

    // 6) Receive
    int recvbuflen = DEFAULT_BUFLen;
    char recvbuf[DEFAULT_BUFLen] = "";

    do { // Receive until the peer closes the connection
        iResult = recv(ConnectSocket, recvbuf, recvbuflen, 0);
        if (iResult > 0) {
            wprintf(L"Bytes received: %d\n", iResult);
            for (int jj = 0; jj < recvbuflen; jj++) {
                cout << recvbuf[jj];
            }
        }
    } while (iResult > 0);

    // 7) close the socket
    iResult = closesocket(ConnectSocket);
    WSACleanup();

    return 0;
}
```

see also: <https://stackoverflow.com/questions/39931347/simple-http-get-with-winsock>
<https://stackoverflow.com/questions/6686261/what-at-the-bare-minimum-is-required-for-an-http-request#9734866>

Server Setup: Specify Connection



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Initialize Winsock [..]
    // Initialize Server Socket [..]
    // Specify Connection
    struct sockaddr_in serverService;
    serverService.sin_addr.s_addr = htonl(INADDR_ANY);
    serverService.sin_family = AF_INET;
    serverService.sin_port = htons(26000);

    return 0;
}
```

Specify Connection

AF_INET: Internet Protocol version 4 (IPv4) address family

INADDR_ANY: allows inbound connections on any local IPv4 address

Port: 26000 (registrierter Port)



<https://beej.us/guide/bgnet/html/split/system-calls-or-bust.html#socket>

<https://stackoverflow.com/questions/21640446/winsock-bind-address-inaddr-any-clarification#21640643>

Server Setup: Bind



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

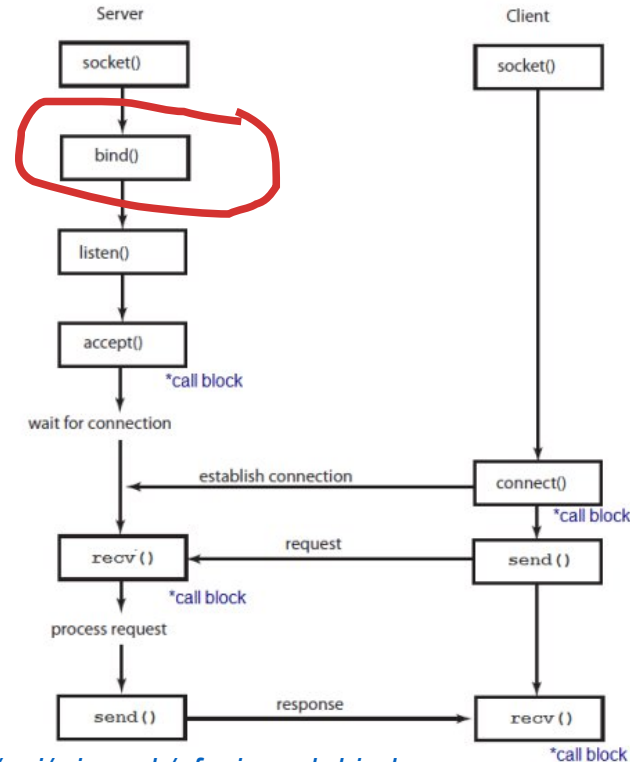
#pragma comment(lib, "Ws2_32.lib")

using namespace std;

int main() {
    // Initialize Winsock [...]
    // Initialize Server Socket [...]
    // Specify Connection [...]
    // Binding the Server Socket
    iResult = bind(ConnectSocket, (struct sockaddr*)&serverService, sizeof(serverService));
    if (iResult != 0) {
        wprintf(L"bind failed with error: %d\n", WSAGetLastError());
    }
    else {
        printf("bind -done-\n");
    }
}
```

Bind

Once you have a socket, associate that socket with a port on your local machine. (e.g. for incoming connections on a specific port — multiplayer network games “connect to 192.168.5.10 port 3490”.)



Server Setup: Listen



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

#pragma comment(lib, "Ws2_32.lib")

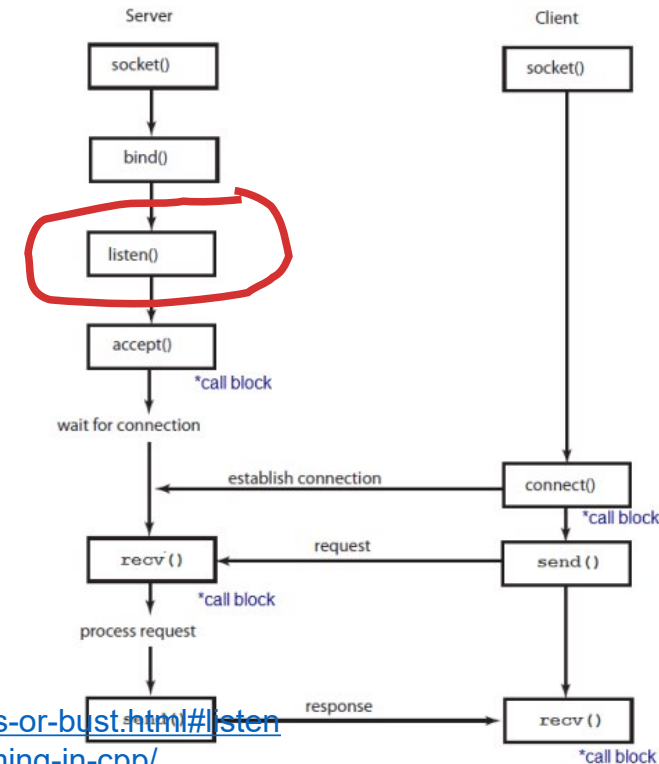
using namespace std;

int main() {
    // Initialize Winsock [...]
    // Initialize Server Socket [...]
    // Specify Connection [...]
    // Binding the Server Socket [...]
    // Listening for Connections
    iResult = listen(ConnectSocket, SOMAXCONN);
    if (iResult != 0) {
        wprintf(L"listen failed with error: %d\n", WSAGetLastError());
        return -1;
    }
    else {
        printf("listen -done-\n");
    }
}
```

Listen

Listen for incoming connections.

2nd parameter: the number of connections allowed on the incoming queue.



<https://beej.us/guide/bgnet/html/split/system-calls-or-bust.html#listen>

<https://www.geeksforgeeks.org/socket-programming-in-cpp/>

<https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-listen>

Server Setup: Accept



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

#pragma comment(lib, "Ws2_32.lib")

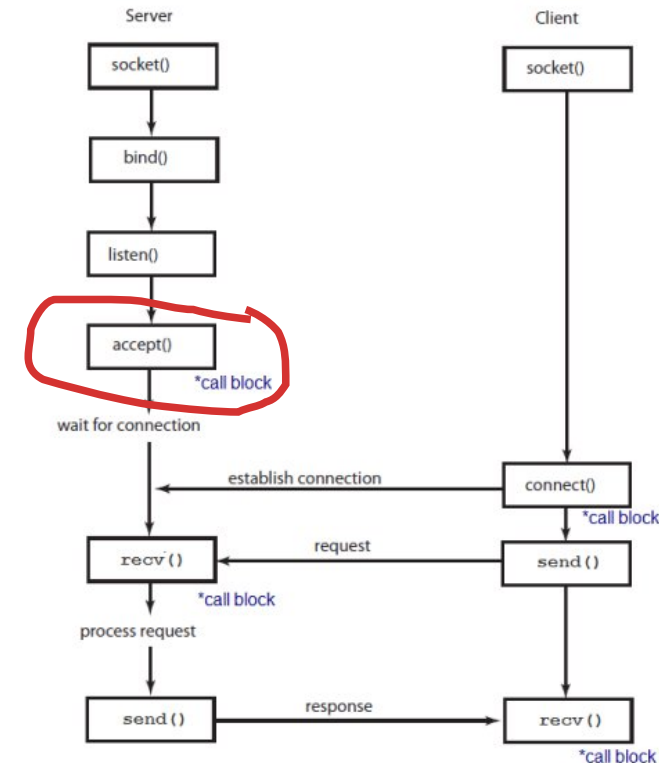
using namespace std;

int main() {
    // Initialize Winsock [...]
    // Initialize Server Socket [...]
    // Specify Connection [...]
    // Binding the Server Socket [...]
    // Listening for Connections [...]
    // Accepting a Client Connection
    int clientSocket = accept(ConnectSocket, nullptr, nullptr);
    if (clientSocket == INVALID_SOCKET) {
        printf("accept failed with error: %d\n", WSAGetLastError());
        closesocket(*ServerConnectSocket);
        WSACleanup();
        return 1; }
    else {
        printf("accept -done-\n");
    }
}
```

Accept

Client connections will be queued up waiting to be accept()ed. You call accept() and you tell it to get the pending connection. It'll return to you a *brand new socket file descriptor* to use for this single connection!

The original one is still listening for more new connections, and the newly created socket is finally ready to send() and recv().



Server Setup: Accept



```
#include <iostream>

#include <winsock2.h>
#include <ws2tcpip.h> // for more recent InetPton

#pragma comment(lib, "Ws2_32.lib")

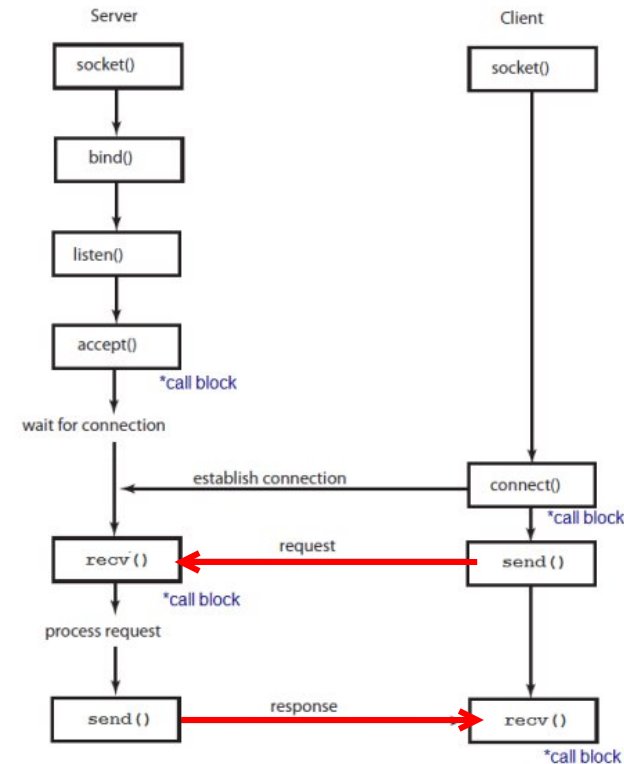
using namespace std;

int main() {
    // Initialize Winsock [...]
    // Initialize Server Socket [...]
    // Specify Connection [...]
    // Binding the Server Socket [...]
    // Listening for Connections [...]
    // Accepting a Client Connection
    int clientSocket = accept(ConnectSocket, nullptr, nullptr);
    if (clientSocket == INVALID_SOCKET) {
        printf("accept failed with error: %d\n", WSAGetLastError());
        closesocket(*ServerConnectSocket);
        WSACleanup();
        return 1; }
    else {
        printf("accept -done-\n");
    }
}
```

Accept

Client connections will be queued up waiting to be accept()ed. You call accept() and you tell it to get the pending connection. It'll return to you a *brand new socket file descriptor* to use for this single connection!

The original one is still listening for more new connections, and the **newly created socket is finally ready to send() and recv()**.



C++ Implementation: Client / Server



```
#include <iostream>
#include <string>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "Ws2_32.lib")
#define DEFAULT_BUFLen 512

using namespace std;

int main() {
    // 1) Startup Winsock Application WSA
    WORD wVersionRequested;
    WSADATA wsaData; // WSADATA data structure
    wVersionRequested = MAKEWORD(2, 2);
    int err = WSAStartup(wVersionRequested, &wsaData);

    // 2) Initialize Socket
    SOCKET ConnectSocket = INVALID_SOCKET;
    ConnectSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    // 3) Specify Connection
    struct sockaddr_in clientService;
    inet_pton(AF_INET, "127.0.0.1", &clientService.sin_addr);
    clientService.sin_family = AF_INET;
    clientService.sin_port = htons(80);

    // 4) Connect to server.
    int iResult;
    iResult = connect(ConnectSocket, (SOCKADDR*)&clientService, sizeof(clientService));

    // 5) Send
    const char* rq_all = "Hallo Server\n";
    iResult = send(ConnectSocket, rq_all, (int)strlen(rq_all), 0);

    // 6) close the socket
    iResult = closesocket(ConnectSocket);
    WSACleanup();

    return 0;
}
```

Further Reading: <https://medium.com/@tharunappu2004/creating-a-simple-tcp-server-in-c-using-winsock-b75dde86dd39>

CLIENT

C++ Implementation: Client / Server



```
#include <iostream>
#include <string>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "Ws2_32.lib")
#define DEFAULT_BUFLen 512

using namespace std;

int main() {
    // 1) Startup Winsock Application WSA
    WORD wVersionRequested;
    WSADATA wsaData; // WSADATA data structure
    wVersionRequested = MAKEWORD(2, 2);
    int err = WSAStartup(wVersionRequested, &wsaData);

    // 2) Initialize Socket
    SOCKET ConnectSocket = INVALID_SOCKET;
    ConnectSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    // 3) Specify Connection
    struct sockaddr_in serverService;
    serverService.sin_addr.s_addr = htonl(INADDR_ANY);
    serverService.sin_family = AF_INET;
    serverService.sin_port = htons(26000);
```

```
// 4) Binding the Server Socket
int iResult = bind(ConnectSocket, (struct sockaddr*)&serverService, sizeof(serverService));

// 5) Listening for Connections
iResult = listen(ConnectSocket, SOMAXCONN);

// 6) Accepting a Client Connection
int clientSocket = accept(ConnectSocket, nullptr, nullptr);

// 7) Receiving Data from the Client
char buffer[1024] = { 0 };
do {
    iResult = recv(clientSocket, buffer, sizeof(buffer), 0);
    if (iResult > 0) {
        wprintf(L"Bytes received: %d\n", iResult);
        cout << "Message from client: " << buffer << endl;
    }
} while (true);

// 8) close the socket
closesocket(*ServerConnectSocket);
return 0;
}
```

Further Reading: <https://medium.com/@tharunappu2004/creating-a-simple-tcp-server-in-c-using-winsock-b75dde86dd39>

SERVER

C++ Implementation LINUX (z.B. Client)



```
#include <iostream>
#include <sys/socket.h>
#include <arpa/inet.h> // inet_aton and inet_pton
#include <string>

using namespace std;

int main() {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serv_addr;
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(26000);
    inet_pton(AF_INET, "192.168.0.93", &serv_addr.sin_addr);
    if (connect(sockfd, (struct sockaddr *) &serv_addr,
        sizeof(serv_addr)) < 0) cout << "ERROR connecting\n";

    string message = " -- test string - \n";
    int iResult = send(sockfd, message.c_str(), message.length(), 0 );
    if (iResult == SO_ERROR) {
        cout << "send failed\n";
    } else {cout << "send " << iResult << "bytes \n";}

    cout << "Toll gemacht\n" ;
}
```

Winsock wurde 1992 von Microsoft entwickelt.

Dazu wurden praktisch alle Funktionen von den Betriebssystemen Unix und BSD übernommen....



C++ Implementation LINUX/MAC (z.B. Client)



```
#include <iostream>
#include <sys/socket.h>
#include <arpa/inet.h> // inet_aton and inet_pton
#include <string>
#include <unistd.h> // close(socket)
using namespace std;

int main() {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serv_addr;
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(26000);
    inet_pton(AF_INET, "192.168.0.93", &serv_addr.sin_addr);
    if (connect(sockfd, (struct sockaddr *) &serv_addr,
        sizeof(serv_addr)) < 0) cout << "ERROR connecting\n";

    string message = "-- test string - \n";
    int iResult = send(sockfd, message.c_str(), message.length(), 0 );
    if (iResult == SO_ERROR) {
        cout << "send failed\n";
    } else {cout << "send " << iResult << "bytes \n";}

    close(sockfd)
    cout << "Toll gemacht\n" ;
}
```





Internetkommunikation: ISO/OSI-Referenzmodell



OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA TLS, SOCKS
Darstellung (6)		
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

Internetkommunikation: ISO/OSI-Referenzmodell



OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA
Darstellung (6)		TLS, SOCKS
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

Internetkommunikation: ISO/OSI-Referenzmodell



OSI-Schicht	TCP/IP-Schicht	Beispiel
Anwendungen (7)	Anwendungen	HTTP, UDS, FTP, SMTP, POP, Telnet, DHCP, OPC UA
Darstellung (6)		TLS, SOCKS
Sitzung (5)		
Transport (4)	Transport	TCP, UDP, SCTP
Vermittlung (3)	Internet	IP (IPv4, IPv6), ICMP (über IP)
Sicherung (2)	Netzzugang	Ethernet, Token Bus, Token Ring, FDDI
Bitübertragung (1)		

(Application Layer)
(Presentation Layer)
(Session Layer)
(Transport Layer)
(Network Layer)
(Data Link Layer)
(Physical Layer)

<https://de.wikipedia.org/wiki/Internetprotokollfamilie>
<https://de.wikipedia.org/wiki/OSI-Modell>

C++ Implementation HTTP Request: ÜBUNG!



```
#include <iostream>
```

```
//#define _WINSOCK_DEPRECATED_NO_WARNINGS  
#include <winsock2.h>  
#include <ws2tcpip.h> // for more recent InetPton  
// #include <stdio.h>
```

```
#pragma comment(lib, "Ws2_32.lib") // C:\Program
```

```
#include <string>
```

```
#define DEFAULT_BUFLen 512  
#define DEFAULT_PORT 27015
```

```
using namespace std;
```

```
// 1) Start Socket with https://learn.microsoft.com/en-us/windows/win32/networking  
// 2) do loopback on localhost  
// - https://learn.microsoft.com/en-us/windows/win32/networking  
// - https://www.codeproject.com/Articles/412511/Socket-Programming-Example  
// 3) Webserver -> HTTP requests  
// - https://www.mediaevent.de/tutorial/http-request.html  
// - Check das mit Tryhackme.com (da noch mal das Tutorial machen und für Vorlesung verwenden!...
```

```
int ConnectFirstWWW() {
```

```
    // 1) Startup Winsock Application WSA  
    WORD wVersionRequested;  
    WSADATA wsaData; // WSADATA data structure  
    int err;
```

```
    cout << "let's test the winsock" << endl;  
    wVersionRequested = MAKEWORD(2, 2);  
    err = WSASStartup(wVersionRequested, &wsaData); // param: requested socket version // return: Info into provided WSADATA data structure
```

C++ Implementation File Transfer: ÜBUNG!



Message from client: -- test Øñ Header ...

```
#include <iostream>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h> // inet_aton and inet_pton
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (sockfd < 0)
```

```
        cout << "ERROR opening socket\n";
```

```
    else
```

```
        cout << " Socket opened .... \n";
```

```
    int portno = 26000;
```

```
    struct sockaddr_in serv_addr;
```

```
    struct hostent* server;
```

```
    serv_addr.sin_family = AF_INET;
```

```
    serv_addr.sin_port = htons(portno);
```

```
    const char *IP = "192.168.0.93";
```

```
    //serv_addr.sin_addr.s_addr = inet_addr(IP);
```

```
    //inet_aton(IP, &serv_addr.sin_addr); // store IP in serv_addr -WORKING-
```

```
    inet_pton(AF_INET, "192.168.0.93", &serv_addr.sin_addr);
```

```
    //serv_addr.sin_addr = htonl() // we are here
```

```
    if (connect(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
```

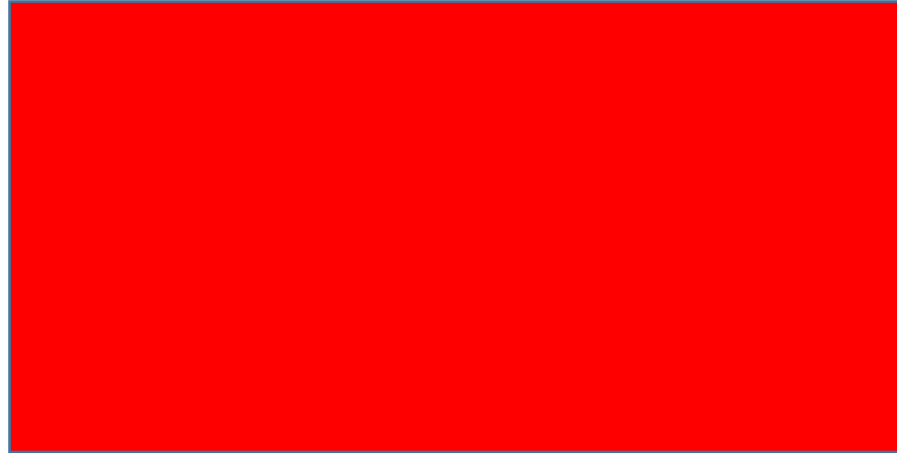
```
        cout << "ERROR connecting\n";
```

```
    else
```

```
        cout << "connection established\n";
```



C++ Implementation: P2P —> PROJEKT



Further Reading:

<https://stackoverflow.com/questions/2843277/c-winsock-p2p>

<https://learn.microsoft.com/en-us/windows/win32/p2psdk/portal>